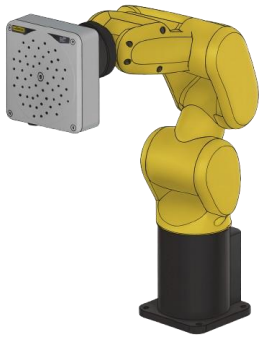

# Robotic Leak Detection System

*Integrating Robotics, PLC Automation and SCADA Visualisation*

## Project Engineering Report

B.Eng. (Hons) in Software and Electronic Engineering
Atlantic Technological University Galway


**Jamie Montgomery**

Student Number: G00419525

Supervisor: Fergus Downey

Industry Partner: Trane Technologies, Galway


Academic Year: 2025/2026

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

# Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University, Galway.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced accordingly throughout this report.

Signed: *Jamie Montgomery* Date: 27/04/2026

Jamie Montgomery.

## Acknowledgements

I would like to thank my project supervisor, Fergus Downey, at ATU Galway for his guidance and support throughout the duration of this project.

I would like to express my appreciation to the Automation Manufacturing Technology (AMT) engineering team at Trane Technologies, Galway, for providing full access to the lab environment, industrial hardware, VPN connectivity to the gateway, and ongoing technical guidance. In particular, Keith Connaughton (AMT Manager), Evan Pierce (Senior Brazing Automation Engineer), & Sean Igoe (Control Systems Engineer Intern) for their ongoing support with overcoming challenges.

Special thanks to the Fluke R&D engineering team, particularly Martin Budweg (R&D Manager), for their direct technical consultation regarding the SV600 acoustic sensor coordinate system and beamforming output. This meeting shaped the direction of the coordinate mapping research.

# Table of Contents

Declaration ..... 2
Acknowledgements ..... 3
Table of Contents ..... 4
Summary ..... 6
Poster ..... 8
Introduction ..... 9
Background & Research ..... 11
Acoustic Leak Detection Theory ..... 11
Field Research - Fluke SV600 Testing (Semester 1, Week 2) ..... 12
Hardware Selection & Specifications ..... 15
Software Platforms ..... 18
Communication Protocols ..... 20
Project Planning & Management ..... 24
Requirements Gathering ..... 24
System Workflow Design ..... 24
Architecture Design ..... 26
Jira Project Management ..... 27
System Architecture ..... 28
Overview ..... 28
Communication Paths ..... 28
IP Address Configuration ..... 29
Implementation - Semester 1 ..... 30
PLC Configuration ..... 30
Robot Programming ..... 32
EtherNet/IP DI/DO Integration ..... 34
UOP (User Operator Panel) Configuration ..... 35
Ladder Logic Development ..... 36
MySQL Database Initial Setup ..... 38
Ignition Frontend - Initial Development ..... 39
Implementation - Semester 2 ..... 41
Fluke SV600 REST API Research & Validation ..... 41
Image Capture Pipeline Development ..... 42
Zero-Byte JPEG Bug - Investigation & Resolution ..... 43

Database Schema Evolution ..... 46
Event Metadata Extraction ..... 47
QR Code Traceability ..... 47
PASS/FAIL Classification ..... 50
Ignition UI - Leak Visualisation Review ..... 50
Operator Override & Notes Feature ..... 51
Fluke SV600 Coordinate System Research ..... 52
Testing & Validation ..... 53
EtherNet/IP Communication Validation ..... 53
Image Capture Pipeline Validation ..... 53
QR Decode Pipeline Validation ..... 53
End-to-End System Test ..... 55
Ethics & Sustainability ..... 56
Results ..... 57
System Performance Summary ..... 57
Technical Achievements ..... 57
Challenges & Solutions ..... 58
Challenge 1: EtherNet/IP I/O Word Mismatch ..... 58
Challenge 2: Zero-Byte JPEG BLOB in Jython ..... 59
Challenge 3: ZXing Java Library Deployment ..... 59
Challenge 4: Fluke SV600 Coordinate System Clarification ..... 60
Conclusion ..... 62
Project Summary ..... 62
Reflection ..... 62
Future Work ..... 63
References ..... 64

## Summary

This report documents the research, design, development, implementation, and testing of an Automated Robotic Leak Detection System created in partnership with Trane Technologies, Galway, as a final year industry project in Software and Electronic Engineering.

The project addresses a persistent quality control problem at Trane Technologies: the manual acoustic inspection of brazed copper joints in HVAC coil assemblies. Prior to this project, the inspection process was entirely manual, operator-dependent, conducted in a hazardous environment, and produced no digital record of results. Undetected leaks at this stage of production result in expensive rework, product failures in the field, and wasted refrigeration.

The solution integrates five core components into a single automated pipeline using a FANUC LR Mate 200iD 6-axis industrial robot, an Allen-Bradley ControlLogix PLC (1756-L71 with 1756-EN2T EtherNet module), a Fluke SV600 acoustic imaging sensor, Ignition SCADA as the central orchestration platform, and a MySQL database for persistent result storage and historical data review.

The system operates as follows: the operator initiates a test cycle from the Ignition HMI by pressing Reset and then Start. Ignition writes a Start tag to the PLC through the ControlLogix EtherNet/IP driver. The PLC validates that the robot is in the home position, enables the UOP (User Operator Panel) signals over EtherNet/IP, and the robot begins its scan cycle. The robot first moves to a QR code scan position where Ignition triggers the Fluke SV600 camera using REST API, captures a JPEG image, saves it to MySQL, retrieves it, and decodes the QR content using the ZXing Java library to extract the product serial number. At this point, a unique capture group UUID (Universally Unique Identifier) is also generated. Both the decoded serial number and the UUID are stored in Ignition memory tags, where they persist for the duration of the test cycle. Once the QR code is received, the operator triggers the nitrogen to pressurise the HVAC coil then the robot moves to Position 1, Position 2, and Full Field-of-View positions. At each position, a Digital Output signal from the robot triggers a PLC tag change that Ignition detects, causing it to call the Fluke REST API, capture the acoustic heatmap image, extract dB

SPL and X/Y/Z coordinate metadata, and store all results to MySQL tagged with the serial number and shared UUID. Each result is automatically classified as PASS or FAIL based on a configurable dB SPL threshold, with the operator able to manually override any classification in the Ignition results dashboard. Once the Full Field-of-View capture is complete and saved, both memory tags are cleared to prevent the serial number and UUID from carrying over into the next test cycle.

All primary scripting was written in Jython (Python on a Java Virtual Machine) within the Ignition SCADA gateway environment. The system was validated on HVAC coil products at Trane Technologies pressurised with nitrogen to simulate operational conditions and reveal potential leak paths.

Key technical challenges overcome during implementation included an EtherNet/IP I/O word mismatch between the robot and PLC causing communication faults, a Jython and Java bug silently filling captured JPEG images with zero bytes, an IT-restricted Java library deployment for the ZXing QR decoding library, and a Gateway Tag Change event script failing to call a gateway-scoped Project Library function due to incorrect script scope configuration.

# Poster

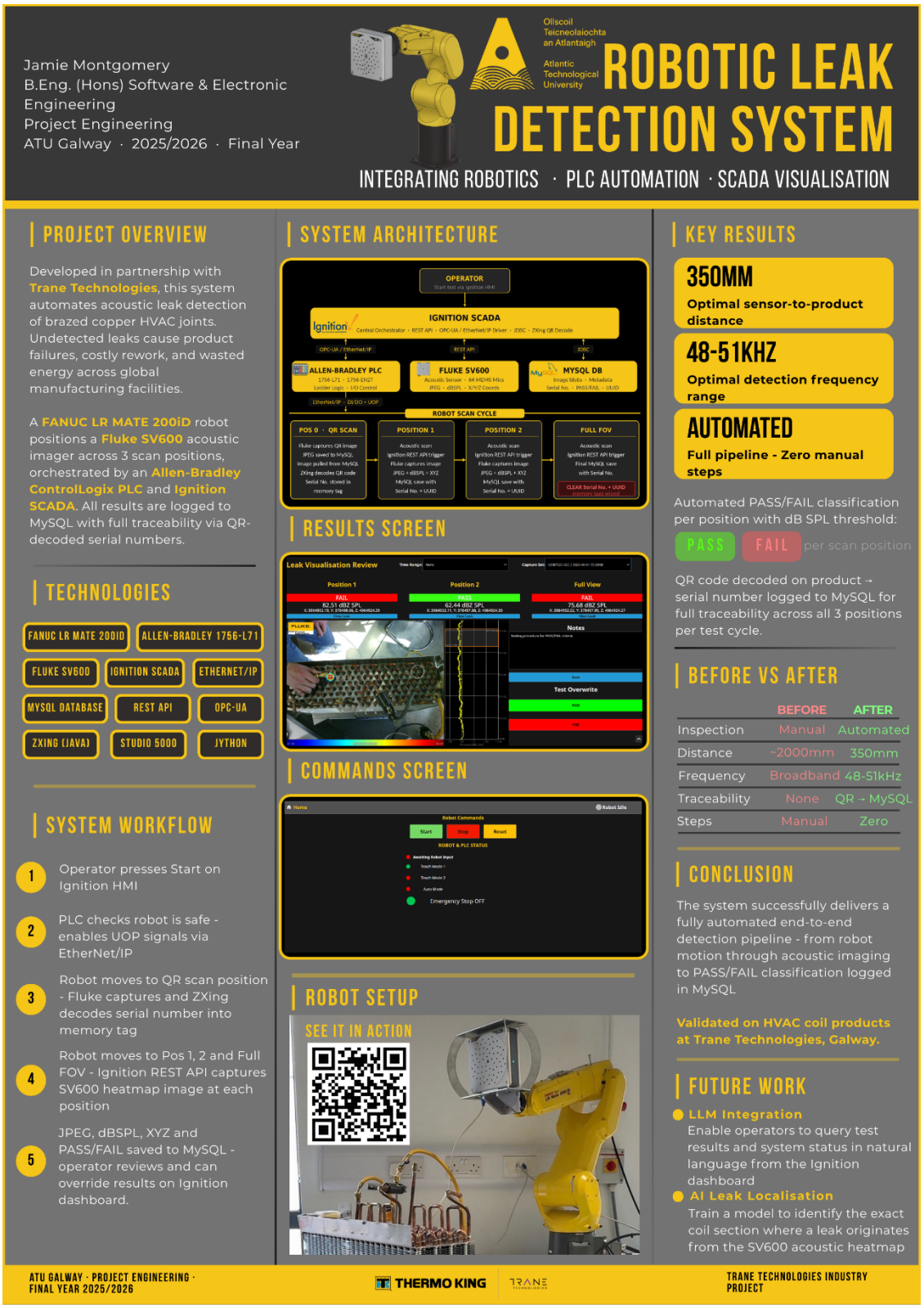


*Figure 1: Project Poster*

# Introduction

Trane Technologies is a global climate innovation company specialising in the manufacture of HVAC systems for residential, commercial, and industrial applications. Across their manufacturing facilities, HVAC coil assemblies are produced by brazing copper pipe joints together. Brazing is a metal joining process that uses a filler metal with a melting point below that of the base materials. When performed correctly, the braze joint forms a hermetically sealed connection capable of maintaining pressure under operational conditions. When a brazing defect occurs, it creates a leak path through which pressurised refrigerant or air can escape. Nitrogen is commonly used as the test gas during inspection because it is chemically inert and will not contaminate the refrigerant circuit. Even small defects are detectable using nitrogen, as it can escape through leak paths as small as $10^{-3}$ to $10^{-6}$ mbarL/s (millibar litres per second).

These leaks, if undetected before product shipment, result in field failures, warranty claims, and significant energy waste as refrigerant escapes from the system. The traditional inspection method at Trane Technologies involved an operator manually scanning the coil assembly using a handheld ultrasonic inspection device (the UE Systems Ultraprobe® 9000) at a fixed inspection station along the manufacturing line while it was pressurised with helium. Helium can detect finer leak paths down to $10^{-9}$ to $10^{-12}$ mbarL/s. Helium is a smaller molecule than nitrogen and therefore escapes through defects more easily, but it is significantly more expensive, requires careful handling, and creates a more hazardous working environment for operators. This manual approach was also time-consuming, inconsistent between operators, produced no digital record of results, and could not be easily reproduced or audited.

This project replaces both the manual process and the use of helium with an automated system using the Fluke SV600 acoustic imager and nitrogen as the test gas. Nitrogen is safer, cheaper, and more readily available than helium, and when used at the correct pressure with the SV600 at the optimal detection distance and frequency range, it provides reliable and repeatable acoustic leak localisation.

The concept for this project originated during an eight-month internship at Trane Technologies prior to final year. During the internship, the acoustic leak detection process

was identified as a strong candidate for automation. At the time, engineers at Trane were evaluating the Fluke SV600 sensor by testing its capabilities at a fixed position. The proposed production approach under consideration involved deploying multiple sensors in parallel to cover the full inspection area of the coil, however at approximately €15,000 per unit, this represented a significant capital investment.

During the internship, I identified an alternative approach. Rather than deploying multiple fixed sensors, a single SV600 mounted to an industrial robot could be repositioned dynamically to cover the same inspection area at around half of the cost. The inspection path is repetitive and predictable, the Fluke SV600 exposes REST API endpoints for programmatic control, and industrial robotics could position the sensor consistently at the optimal detection distance across multiple scan positions. This formed the basis for the project proposal.

The objective of this project is to design and implement a complete end-to-end automated leak detection system that replaces the manual inspection process. The system must automatically position the acoustic sensor, capture inspection data, log results with full product traceability, classify each result as PASS or FAIL, and present results to an operator through a SCADA interface. The solution must be implemented using the industrial hardware already present at the Trane Technologies AMT lab to maximise the potential for eventual production deployment.

The scope of the project encompasses hardware integration (**robot, PLC, sensor**), industrial communication protocols (**EtherNet/IP, OPC-UA, REST API, JDBC**), software development (**Jython scripting in Ignition SCADA, ladder logic in Studio 5000, MySQL database design**), and a user-facing SCADA interface for result visualisation and operator interaction.

# Background & Research

## Acoustic Leak Detection Theory

Acoustic leak detection is a non-contact inspection technique that exploits the ultrasonic noise generated when pressurised gas escapes through a small hole or defect. When a leak occurs, the escaping gas undergoes turbulent expansion as it passes from high pressure to low pressure. This turbulence generates broadband noise that extends well into the ultrasonic frequency range, typically above 20kHz, which is beyond the range of human hearing (humans can typically hear up to 20kHz).

Acoustic imaging devices use an array of microphones to detect this ultrasonic noise and apply beamforming signal processing algorithms to determine the direction from which the sound is arriving. Beamforming works by combining the signals from multiple microphones with calculated time delays, constructively reinforcing signals arriving from a specific direction while cancelling signals from other directions. By scanning multiple directions, the system builds a spatial map of acoustic intensity, which is then overlaid on a visible-light camera image to produce an acoustic heatmap.

Different gases have different molecular sizes and therefore produce different acoustic signatures when leaking. Air and nitrogen are commonly used as test gases because they are safe, inexpensive, and widely available. Nitrogen is preferred in refrigeration applications because it is chemically inert and will not contaminate the refrigerant circuit.

Key parameters that affect acoustic leak detection accuracy include detection distance (closer is better, as acoustic energy dissipates with distance), frequency range (higher frequencies are more directional but have shorter range), and environmental noise (background ultrasonic noise from manufacturing equipment can create false positives if not filtered). In the context of this project, these principles are applied using the Fluke SV600 acoustic imager, which uses a 64-element MEMS (Micro-Electro-Mechanical Systems) microphone array and beamforming to localise leaks in pressurised HVAC coil assemblies.

## Field Research - Fluke SV600 Testing

Before designing the automated system, field research was done at the Trane Technologies manufacturing facility in Kolin, Czech Republic, where HVAC coils are produced before being shipped to ThermoKing, Galway for integration into truck refrigeration units. This research characterised the performance of the Fluke SV600 acoustic sensor on the target HVAC coil products, with the aim of establishing the optimal detection parameters that the robot would need to replicate consistently and validating the sensor's suitability for the application.

### Initial Setup

The initial setup of the Fluke SV600 involved enabling Power over Ethernet (PoE) as the device power source and resolving an IP address conflict where the device had been left in static mode by a previous user. A custom network environment was configured to match the device static IP, allowing access to the web interface through a browser. Once accessible, the device IP was changed to the correct gateway address for the lab network.

Ground markings were made at distances of 1400mm, 1000mm, and 600mm from the product surface to enable systematic distance testing. A pressurised HVAC coil with a known leak was used as the test subject. Testing was also conducted on 40 different HVAC coils with unknown leak status. Nitrogen pressure was increased gradually to determine at what pressure the SV600 would begin detecting the leak.

### Distance Testing

Testing at 1400mm distance with approximately 7 bar (101 PSI) of nitrogen pressure produced a visible response but with poor localisation accuracy. The acoustic heatmap showed energy spread across a wide area rather than pinpointing the precise leak location. This was attributed to two factors: the longer path length reduces signal intensity, and at greater distances the angular resolution of the beamforming algorithm results in a larger physical area being highlighted.

At 600mm distance, a significant improvement in localisation accuracy was observed. Adjusting the blob size slider on the SV600 interface to a small setting further improved the precision of the highlighted area. The blob size parameter controls the minimum

acoustic source size that the device will attempt to localise, with a smaller setting filtering out diffuse background noise.

A key observation was that higher nitrogen pressure (above approximately 5 bar) sometimes caused acoustic reflections off adjacent metal surfaces, causing the device to show the leak at an incorrect location. This was ricocheting, it is a fundamental limitation of acoustic imaging in environments with reflective surfaces. Pinning the minimum pressure while leaving the maximum auto-ranging helped to show only the constant, primary leak source rather than reflected signals.

Overall, any detection distance of 600mm or below produced consistent and reliable localisation results, with 350mm identified as the optimal operating distance for the automated system due to the robot's ability to position the sensor precisely and repeatably at that range.

## Frequency Range Testing

The most significant improvement in detection accuracy came from adjusting the operating frequency range. The SV600 allows the user to select a frequency band within the range of 2-52kHz, acting as a configurable bandpass filter that restricts acoustic analysis to the selected range and rejects background noise outside that band. Testing showed that a narrow band of 48-51kHz provided the best balance between background noise rejection and detection sensitivity for this specific application.

At higher frequencies, background noise from other manufacturing equipment is filtered out, but the detection range is reduced because higher frequency ultrasound attenuates more rapidly with distance. This finding directly justified the use of a robot arm - by positioning the sensor at 350mm from the product at 48-51kHz, the system could achieve both the noise rejection of high-frequency operation and the sensitivity of close-range detection.

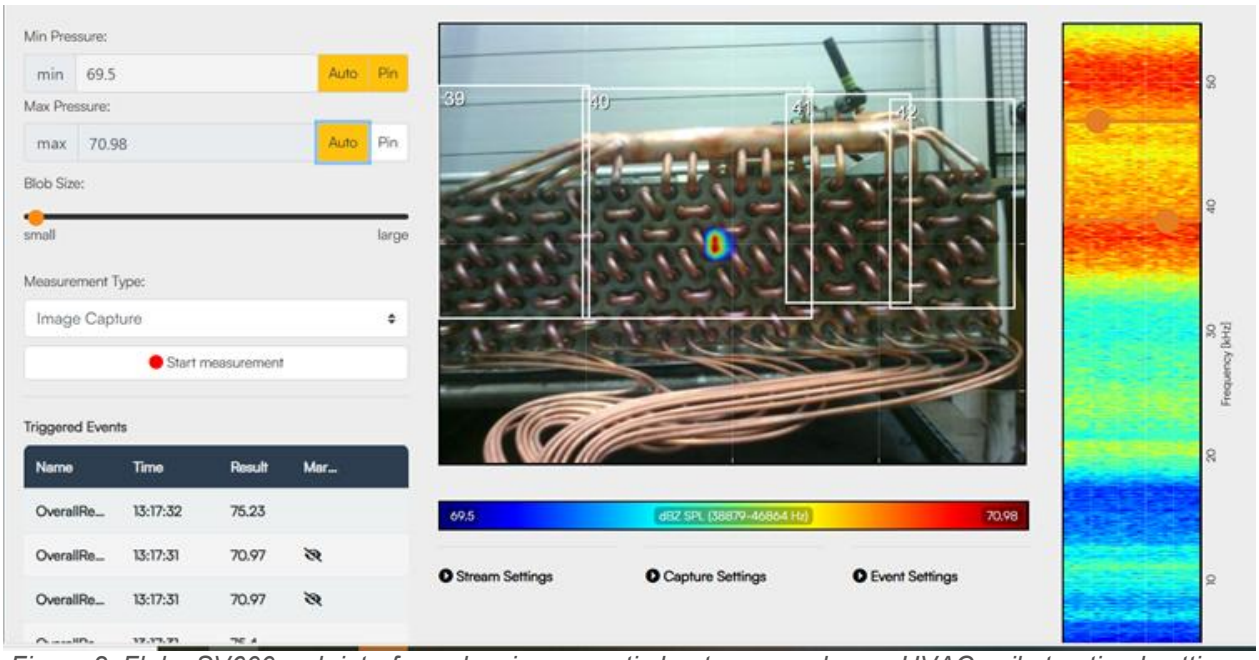


*Figure 2: Fluke SV600 web interface showing acoustic heatmap overlay on HVAC coil at optimal settings (48-51kHz, 600mm)*

## Field Research Conclusions

The field research confirmed the following optimal operating parameters for the automated system:

- Detection distance: 350mm
- Frequency range: 48-51kHz (narrow band for noise rejection)
- Blob size: Small (for precise leak localisation)
- Distance setting in device: Must match physical detection distance to avoid offset errors
- Pressure pinning: Pin minimum pressure, leave maximum auto-ranging

These findings directly shaped the robot position register design. The three scan positions were defined so that the Fluke SV600 end-effector is located at exactly 350mm from the product surface at each inspection point. The Full Field-of-View position captures the complete coil from a slightly greater distance to maximise coverage.

## Hardware Selection & Specifications

### FANUC LR Mate 200iD Robot

The FANUC LR Mate 200iD is a compact 6-axis industrial robot. The specific variant in the Trane Technologies AMT lab is the 7L (long reach) variant. It was selected for this project because it was already installed in the lab due to it being a common robot used in Trane manufacturing facilities globally, and its compact form factor is well suited to inspection tasks in confined spaces around coil assemblies.

| Specification | Value |
|---|---|
| Model | LR Mate 200iD/7L |
| Axes | 6 |
| Payload | 7kg |
| Reach | 717mm |
| Repeatability | ±0.02mm |
| Controller | R-30iB Plus |
| Communication | EtherNet/IP Adapter (RTL-R784) |
| Programming | iPendant, TP (Teach Pendant) |
| Installation | Floor mount, angle mount |

*Table 1: FANUC LR Mate 200iD/7L Specifications*

The robot is equipped with a FANUC EtherNet/IP Adapter module which allows it to act as a slave device within the PLC-controlled network. Communication uses a standardised I/O exchange in which the PLC (scanner/master) writes output bits to the robot (adapter/slave), which reads them as input bits, and vice versa. This enables the PLC to send motion commands and the robot to send position status signals in real time

### Allen-Bradley ControlLogix PLC

The Allen-Bradley ControlLogix system is a modular PLC platform widely used in industrial automation. The specific hardware in the Trane Technologies AMT lab consists of a 1756 backplane chassis housing the following modules:

- **1756-L71 ControlLogix Controller:** The CPU module. It executes the ladder logic downloaded from Studio 5000 Logix Designer. It contains 2MB of user memory for logic, tag database, and communication configurations.
- **1756-EN2T EtherNet/IP Module:** The network interface module. It allows the controller to communicate via EtherNet/IP with the FANUC robot, the Ignition SCADA gateway, and any other EtherNet/IP devices on the network.

| Module | Function |
|---|---|
| 1756-L71 | CPU - executes ladder logic, manages tag database |
| 1756-EN2T | EtherNet/IP bridge module for network communication |
| Local:1:C | Local I/O rack configuration |
| Local:11 | Digital Output module (DO signals to robot) |

*Table 2: Allen-Bradley ControlLogix PLC Module Configuration*

## Fluke SV600 Acoustic Sensor

The Fluke SV600 is a fixed-mount acoustic imaging device designed for continuous monitoring applications in industrial environments. It uses a 64-element MEMS (Micro-Electro-Mechanical Systems) microphone array arranged in a sunflower pattern. The sunflower arrangement is specifically optimised for beamforming applications, as it provides uniform spatial sampling across the field of view while minimising grating lobes (false acoustic peaks caused by evenly spaced microphone arrays). The 65° FoV (Field-of-View) and optimal detection range of 350mm informed the robot scan position design, with each position defined to capture half of the coil surface per scan.

| Specification | Value |
|---|---|
| Microphone Array | 64 MEMS digital microphones, sunflower arrangement |
| Frequency Range | 2kHz to 52kHz |
| Camera | Integrated visible light, 720p @ 30fps, 65° FoV |
| Power | Power over Ethernet (PoE+), 100-240V AC via PoE injector |
| Connectivity | RJ45 Ethernet, HTTP REST API on port 9011 |
| Ingress Protection | IP54 |
| Operating Temperature | -20°C to 50°C |
| Physical Size | 170 x 170 x 65mm |
| Weight | 0.85kg |

| Specification | Value |
|---|---|
| API Protocol | HTTP REST, WebSocket (port 9012) |
| Authentication | HTTP Basic Authentication |

*Table 3: Fluke SV600 Technical Specifications*

The SV600 exposes three API levels: the Sound Source Detection API (primary interface for acoustic data), the Authentication API (for access control), and the Device Manager API (for device-level configuration). For this project, the Sound Source Detection API is primarily used, specifically the sound surface capture endpoint and the event results endpoint.

The SV600 was selected for this project because it was already under evaluation by the AMT engineering team, it exposes a built-in HTTP REST API enabling programmatic integration with Ignition SCADA, and its fixed-mount form factor is compatible with end-effector mounting on the FANUC robot.

## Software Platforms

### Ignition SCADA

Ignition by Inductive Automation is an industrial application platform for SCADA (Supervisory Control and Data Acquisition), HMI (Human-Machine Interface), MES (Manufacturing Execution System), and IIoT (Industrial Internet of Things) applications. It runs as a Java-based server application (the Ignition Gateway) and clients connect with a web browser using the Perspective module (mobile-responsive) or through the Ignition Designer desktop application for development.

Ignition was selected for this project for several reasons. It natively supports Allen-Bradley ControlLogix PLCs through the Allen-Bradley Logix Ethernet driver. It supports MySQL database connectivity through its built-in JDBC driver framework. Its scripting environment uses Jython, a Java implementation of Python, which allows direct use of Java libraries such as ZXing for QR decoding. It also provides WebDev and REST capabilities for calling external APIs.

Because Ignition runs on a JVM (Java Virtual Machine), its Jython scripting environment can directly import and use Java classes and libraries. This enabled the use of ZXing Java library for QR code decoding and Java native I/O classes for reliable binary image retrieval from the Fluke's REST API endpoints.

The Ignition gateway runs on a PC within the Trane Technologies AMT lab network. All scripting was developed within the Ignition Designer as project library functions (gateway-scoped) and tag event scripts. Gateway-scoped scripts are essential for this project because they execute on the server rather than on the client, allowing them to make outbound network requests to the Fluke SV600 REST API and perform direct database operations using JDBC without the security restrictions. Tag event scripts, which fire when a PLC tag changes value, also execute in gateway scope – meaning any project library function they call must be also configured as gateway-scoped. If a tag event script attempts to call a Designer-scoped function, it will fail silently at runtime without raising an error, which was a challenge encountered during implementation.

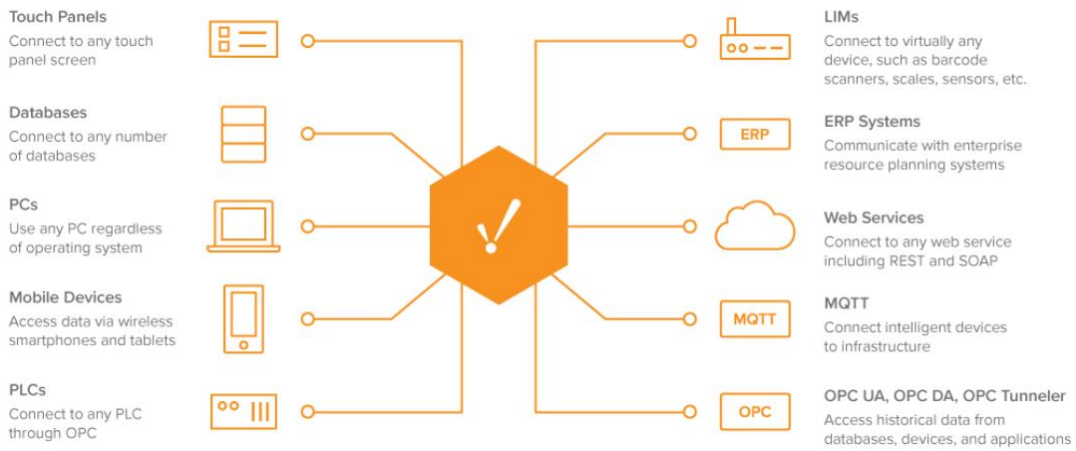


*Figure 4: Ignition SCADA connectivity overview showing all supported integration types*

## Studio 5000 Logix Designer

Studio 5000 Logix Designer is the programming environment for Allen-Bradley ControlLogix PLCs. It is used to create, download, and monitor ladder logic programs on the PLC. For this project, ladder logic was developed to sequence the robot motion, handle start/stop/reset commands from Ignition, manage the UOP signal enabling rungs, and provide status feedback to the SCADA system.

The project uses Logix Designer version 35.11 to match the firmware version on the 1756-L71 controller. An important issue encountered early on was a firmware version mismatch when a different PLC rig was assigned for the project (after the original rig was needed for another project), the firmware on the replacement controller had been updated. This required flashing the firmware back to the original version to maintain compatibility with the developed ladder logic.

## MySQL Database

MySQL was selected as the database platform for this project over Microsoft SQL Server (which was initially prototyped in Semester 1) because it is open-source, widely supported by Ignition's JDBC driver, and sufficient for the data volumes involved in this application. The decision to switch was made in Semester 2 after confirming that the AMT team's existing MySQL server was accessible from the Ignition gateway and avoided the need for IT approval to add a new database server on the network.

The database stores all test result data in a single primary table (capture_images) that iteratively evolved throughout Semester 2 as new features were added. The capture_images table stores the JPEG image blob, capture timestamp, capture position, serial number, capture group UUID, dB SPL measurement, X/Y/Z acoustic coordinates, PASS/FAIL result, and operator notes for each scan position captured during a test cycle. A key design decision was storing JPEG images as LONGBLOB data in the database rather than as file paths. This approach ensures that any computer with access to the Ignition gateway can retrieve and display the images without requiring shared file system access.

## Communication Protocols

### EtherNet/IP

EtherNet/IP (Ethernet Industrial Protocol) is a standard industrial communication protocol that runs over standard Ethernet hardware. It is widely used in Allen-Bradley and other industrial automation systems, like the FANUC robot. EtherNet/IP uses the Common Industrial Protocol (CIP) as its upper-layer protocol, which defines how devices exchange data.

In this project, EtherNet/IP is used for two communication paths: between the Allen-Bradley PLC (scanner) and the FANUC robot (adapter) for real-time I/O exchange, and between the Ignition gateway and the PLC using the Allen-Bradley ControlLogix Ethernet driver for tag read/write operations. The scanner/adapter model means the PLC initiates and manages the I/O connection, polling the robot for input data and sending output data on each RPI (Requested Packet Interval) cycle.

The RPI was configured to 32ms for this project. This means the PLC sends its output bits to the robot every 32ms and reads the robot's input bits every 32ms. This is more than adequate for the position signalling required in this application, where position changes occur over seconds rather than milliseconds. The I/O connection was configured for 8 input bytes and 8 output bytes, equivalent to 4 words of 16 bits each, matching the FANUC robot controller’s EtherNet/IP adapter configuration.

- Rack 89: EthernetIP
- Rack 106: EtherCat

The "Slot" is the location on the rack where the I/O is connected.

| Header | Slot 1 | Slot 2 | Slot 3 | Slot 4 | Slot 5 |
|---|---|---|---|---|---|
| | Module 1 | Module 2 | Module 3 | Module 4 | Module 5 |

*Figure 5: EtherNet/IP rack and slot addressing concept showing rack 89 for EtherNet/IP I/O*

## OPC-UA

OPC-UA (Open Platform Communications Unified Architecture) is a machine-to-machine communication standard used primarily for SCADA and industrial control applications. Unlike EtherNet/IP, which operates at the I/O level, OPC-UA operates at a higher abstraction level, exposing named tags and their values to client applications.

Ignition communicates with the PLC through the Allen-Bradley ControlLogix Ethernet driver. While this driver uses EtherNet/IP at the network level to read and write PLC memory, it exposes the PLC tag database to Ignition through an OPC-UA server. This means Ignition can browse the PLC tag tree, subscribe to tag value changes, and write tag values without any manual data mapping, as the driver reads the tag structure directly from the controller.

Bidirectional tag bindings are required for control tags (HMI_Start, HMI_Stop, HMI_Reset) because Ignition must read the current PLC tag value and write new values when the operator interacts with the HMI buttons. The connection path for the ControlLogix configuration is 1,0, where 1 indicates communication through the backplane and 0 indicates the processer is in slot 0 of the chassis.

## HTTP REST API

The Fluke SV600 exposes its functionality through an HTTP REST (Representational State Transfer) API on port 9011. REST is an architectural style for distributed systems that uses standard HTTP methods (GET, POST, PUT, DELETE) to interact with resources identified by URLs. REST was well suited for this integration because it operates over the existing Ethernet network without requiring additional drivers or middleware, and its JSON responses are directly parseable with Ignition's Jython scription environment using the built-in json module.

The SV600 API uses HTTP Basic Authentication for all requests, requiring a Base64-encoded username:password pair in the Authorisation header of every request. In this project, the factory default credentials (admin:admin) were used, encoded as a Base64 string and passed in the Authorization header of every API call. A reusable Jython function was written to generate this header automatically for all requests rather than hardcoding it repeatedly across multiple scripts.

The SV600 REST API is structured around three document categories: the Sound Source Detection API (primary interface for acoustic data), the Authentication API (for access control), and the Device Manager API (for device-level configuration). All data except captured images is returned in JSON format. Images are returned as binary JPEG data in the response body. For this project, only GET requests are used, one endpoint to capture the acoustic heatmap image and one to retrieve the event metadata including dB SPL measurement and X/Y/Z acoustic source coordinates.

## JDBC

JDBC (Java Database Connectivity) is a Java API that enables Java and JVM-based applications such as Jython scripting environment to execute SQL statements. Ignition uses JDBC drivers to connect to databases, with the database connection configured in the Ignition Gateway web console. Once configured, database operations in Jython scripts are performed using Ignition system functions such as system.db.runQuery() (SELECT), system.db.runPrepUpdate() (INSERT and UPDATE), and system.db.runScalarPrepQuery() (returning a single value).

The MySQL JDBC connection uses the connection name MySQL_COMM throughout all Ignition scripts. Parameterised queries using placeholders (?) are used for all INSERT and UPDATE operations to prevent SQL injection and handle data type conversion cleanly. Retrieving the primary key of each newly inserted image row was essential to this project, as it allowed the event metadata UPDATE to be applied to the exact correct database row using a WHERE id =? clause.

# Project Planning & Management

## Requirements Gathering

The project requirements were established through a series of meetings with the Trane Technologies AMT team early in Semester 1. The initial project concept was presented to the AMT team, who provided feedback on what additional functionality would be valuable from an industry perspective.

Key requirements identified through these meetings included:

- Automated acoustic leak detection for HVAC coils using a robot-guided inspection path
- PLC-coordinated sequencing for robot motion, sensor activation, and test execution
- Data logging and visualisation of results through an Ignition-based dashboard with real-time and historical analytics
- Serial number traceability linking each test result to a specific product
- Safe operation with E-Stops, interlocks, and compliance with industrial safety standards
- Per product data tracking on each product to enable trend analysis and predictive quality control

An important piece of feedback from the AMT engineers was the addition of per-product data tracking, which was not part of the original concept. This feedback directly led to the MySQL database design and QR code traceability feature that became central to the final system.

## System Workflow Design

Following the requirements meetings, a system workflow was designed to define the sequence of operations. The workflow was developed early in Semester 1 and evolved throughout the project. The QR code scan position was added in Semester 2 following AMT team feedback on per-product traceability requirements. The final implemented workflow is as follows:

1. The operator initiates the test cycle from the Ignition HMI by pressing Reset and then Start.
2. Ignition writes a Start tag to the PLC via the Allen-Bradley ControlLogix EtherNet/IP driver.
3. The PLC checks whether the robot is at the home position. If not safe, the PLC sends a fault code tag to Ignition.
4. If safe, the PLC enables the UOP signals and the robot begins its scan cycle.
5. The robot moves to Position 0 (QR scan position). The Fluke SV600 camera captures an image of the QR code, which is saved to MySQL, retrieved, and decoded using ZXing to extract the product serial number. A capture group UUID is generated, and both values are stored in Ignition memory tags.
6. The operator pressurises the HVAC coil with nitrogen before the robot proceeds to acoustic inspection.
7. The robot moves to Position 1. A Digital Output signal triggers a PLC tag change, Ignition detects this and calls the Fluke REST API to capture the acoustic heatmap image.
8. The SV600 returns the JPEG image, dB SPL measurement, and X/Y/Z coordinates. Ignition stores all results to MySQL tagged with the serial number and UUID. A PASS/FAIL result is automatically assigned based on the dB SPL threshold.
9. Steps 7 and 8 repeat for Position 2 and Full Field-of-View.
10. The PLC signals Test Complete. Both memory tags are cleared to prevent carryover into the next test cycle.
11. Ignition displays results on the Leak Visualisation Review dashboard, filterable by serial number and date.

## Architecture Design

The architecture diagram was first developed early in Semester 1 and was iteratively refined throughout both semesters as the system evolved. The initial architecture placed Microsoft SQL Server as the database (later changed to MySQL in Semester 2) and did not include the QR code traceability feature.

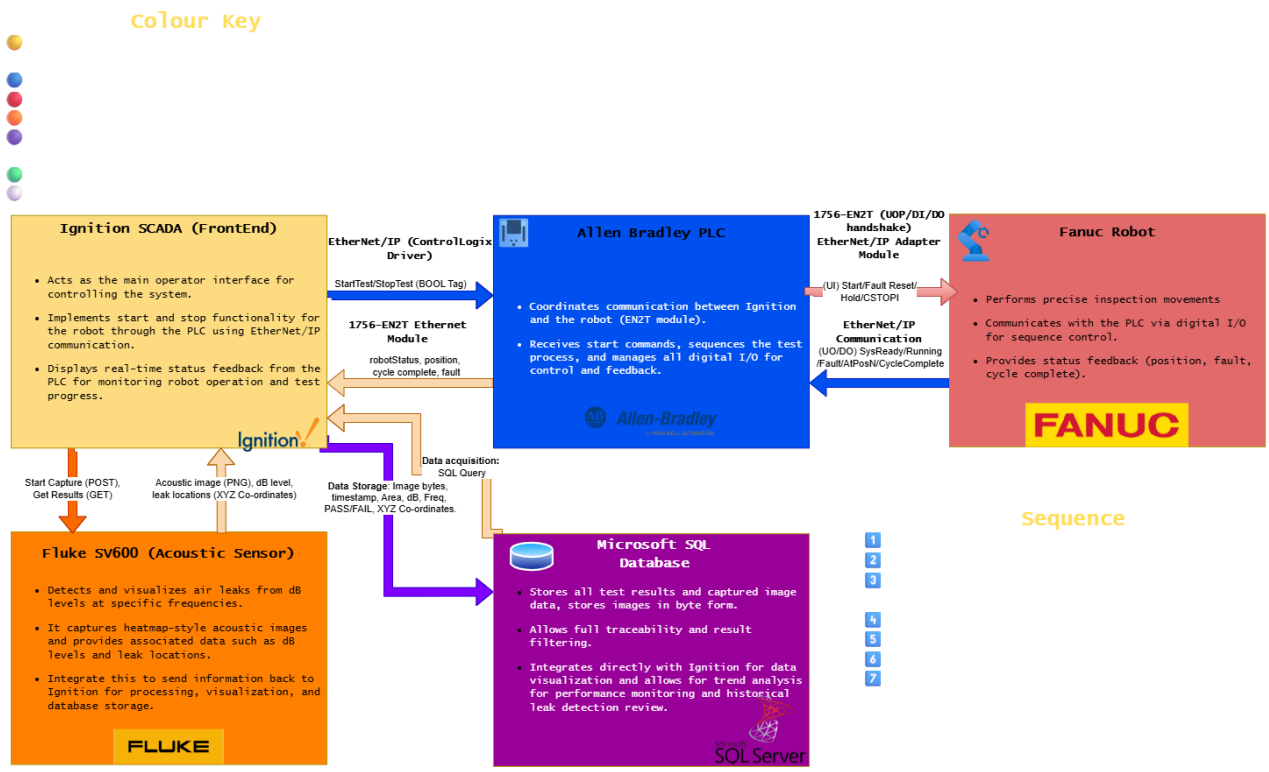


*Figure 6: Original Semester 1 architecture diagram showing system component relationships*

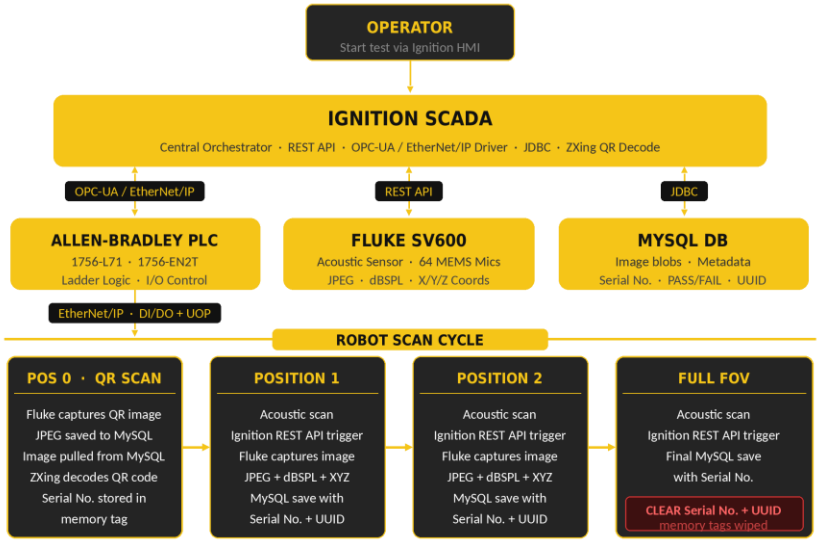


*Figure 7: Final system architecture diagram reflecting all Semester 2 additions*

## Jira Project Management

Jira was used throughout both semesters for sprint planning, task tracking, and progress monitoring. The project was organised into eight epics corresponding to the major development phases:

- FYP-30: Project Proposal (Semester 1, Sprint 1)
- FYP-45: PLC Logic Development
- FYP-46: FANUC Robot Programming
- FYP-47: Ignition SCADA Development
- FYP-48: Fluke API Integration (Semester 2)
- FYP-49: MySQL Database & Data Logging
- FYP-50: Testing
- FYP-51: Reporting & Visualisation
- FYP-52: Documentation & Evaluation

Each epic was broken down into individual tasks and assigned to two-week sprints with roughly 35-40 story points (1-2 hours of work = 1 story point). Sprint planning was reviewed weekly during standup meetings with the project group, and the Jira board was updated to reflect completed tasks, blockers, and upcoming work.

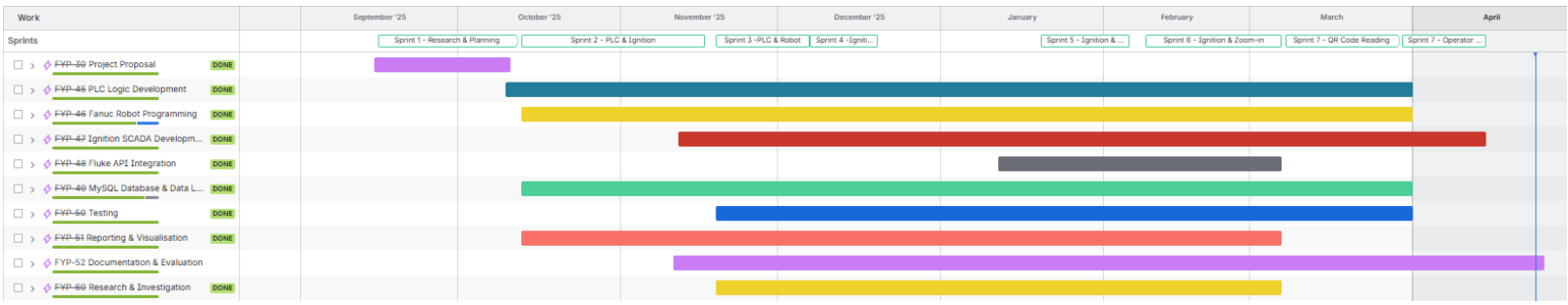


*Figure 8: Jira project timeline (Gantt view) sprint allocation across both semesters*

# System Architecture

## Overview

The system architecture follows a hierarchical integration model with Ignition SCADA functioning as the central orchestration hub. All five components communicate either directly with Ignition or through the PLC, which acts as the sequencing engine and safety interlock layer between Ignition and the robot, ensuring the robot cannot be commanded to move unless the home position is confirmed and all UOP safety signals are active.

The key design principle is that no component communicates directly with another without going through Ignition or the PLC. This ensures a single point of control, predictable sequencing, and a clear data flow that can be monitored and debugged at the Ignition gateway level through the OPC-UA tag browser, gateway logs, and Perspective HMI.

## Communication Paths

The four primary communication paths in the system are:

| Path | Protocol | Purpose |
|---|---|---|
| Ignition ↔ PLC | OPC-UA / EtherNet/IP | Tag read/write: start, stop, reset, position status, fault codes |
| Ignition ↔ Fluke SV600 | HTTP REST API | GET capture trigger, GET image bytes, GET event metadata |
| Ignition ↔ MySQL | JDBC | INSERT image blobs and metadata, SELECT for UI queries |
| PLC ↔ FANUC Robot | EtherNet/IP DI/DO + UOP | Robot start/stop/reset commands and position status signals |

*Table 4: System communication paths and protocols*

## IP Address Configuration

All network devices are connected through the Trane Technologies AMT lab Ethernet switch. These addresses were either statically assigned or confirmed during the initial network setup of the project with close communication with ThermoKings IT department. The IP addresses assigned to each device are:

| Device | IP Address | Role |
|---|---|---|
| Allen-Bradley PLC (1756-EN2T) | 10.188.2.144 | Scanner / EtherNet/IP master |
| FANUC Robot Controller | 10.188.2.186 | EtherNet/IP adapter (slave) |
| Ethernet Switch | 10.188.2.180 | Routes traffic between all lab devices |
| Fluke SV600 | 10.188.8.188 | REST API server on port 9011 |
| Ignition Gateway PC | Local network | SCADA server, OPC-UA client, JDBC client |

*Table 5: Network device IP address configuration*

Connectivity between all devices was verified using ping tests. All devices responded successfully with stable round-trip times, confirming the network path was reliable before integration testing began.

```
PS C:\Users\u127644> ping 10.188.2.188

Pinging 10.188.2.188 with 32 bytes of data:
Reply from 10.188.2.188: bytes=32 time=22ms TTL=57
Reply from 10.188.2.188: bytes=32 time=30ms TTL=57
Reply from 10.188.2.188: bytes=32 time=17ms TTL=57
Reply from 10.188.2.188: bytes=32 time=23ms TTL=57

Ping statistics for 10.188.2.188:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 17ms, Maximum = 30ms, Average = 23ms
PS C:\Users\u127644>
```

*Figure 9: Pinging Fluke SV600 on TK's Network*

# Implementation - Semester 1

## PLC Configuration

PLC development began in Semester 1 after VPN access to the Trane Technologies gateway was approved by IT, enabling remote connection to the PLC from outside the lab. The initial work focused on establishing the connection path between the Ignition gateway and the PLC, then creating the core tag structure.

### PLC Tag Structure

All tags in the PLC controller are structured under the controller tag database in Studio 5000. Tags were created for all robot-to-PLC and PLC-to-robot signals, as well as HMI control inputs and status outputs. The tag naming convention uses the prefix LDRobot_ (Leak Detection Robot) for all robot-related tags:

Controller Tags - jamiesFYP_PLC(controller)

Scope: jamiesFYP_PLC Show: All Tags Enter Name Filter...

| Name | Base Tag | Data Type | Description |
|---|---|---|---|
| HMI_Reset | | BOOL | Ignition HMI Reset |
| HMI_Start | | BOOL | Ignition HMI Start |
| HMI_Stop | | BOOL | Ignition HMI Stop |
| LDRobot_AtFullView | GLW_LAB_LD_ROB01:I1.Input[3].3 | BOOL | Robot at Fluke Full View |
| LDRobot_AtHomePos | GLW_LAB_LD_ROB01:I1.Input[3].0 | BOOL | Robot at Home Pos |
| LDRobot_AtPos1 | GLW_LAB_LD_ROB01:I1.Input[3].1 | BOOL | Robot at Fluke Pos 1 |
| LDRobot_AtPos2 | GLW_LAB_LD_ROB01:I1.Input[3].2 | BOOL | Robot at Fluke Pos 2 |
| LDRobot_AtQRScanPos | GLW_LAB_LD_ROB01:I1.Input[3].5 | BOOL | Robot at Fluke QR Code Pos |
| LDRobot_Auto | GLW_LAB_LD_ROB01:I1.Input[2].4 | BOOL | Robot in AUTO Mode |
| LDRobot_CycleComplete | GLW_LAB_LD_ROB01:I1.Input[3].4 | BOOL | Robot Cycle Complete |
| LDRobot_CycleStop | GLW_LAB_LD_ROB01:O1.Output[0].3 | BOOL | Robot Cycle Stop |
| LDRobot_Enable | GLW_LAB_LD_ROB01:O1.Output[0].7 | BOOL | Robot Enable |
| LDRobot_EStop | GLW_LAB_LD_ROB01:I1.Input[2].7 | BOOL | Robot Emergency Stop Pressed |
| LDRobot_FaultReset | GLW_LAB_LD_ROB01:O1.Output[0].4 | BOOL | Robot Fault Reset |
| LDRobot_Hold | GLW_LAB_LD_ROB01:O1.Output[0].1 | BOOL | Robot Hold |
| LDRobot_IMSTP | GLW_LAB_LD_ROB01:O1.Output[0].0 | BOOL | Robot Immediate Stop |
| LDRobot_ProgRunning | GLW_LAB_LD_ROB01:I1.Input[0].2 | BOOL | Robot Program Running |
| LDRobot_SFSPD | GLW_LAB_LD_ROB01:O1.Output[0].2 | BOOL | Robot Safety Fast Stop Device |
| LDRobot_Start | GLW_LAB_LD_ROB01:O1.Output[0].5 | BOOL | Robot Start |
| LDRobot_SysReady | GLW_LAB_LD_ROB01:I1.Input[0].1 | BOOL | Robot System Ready |
| LDRobot_T1 | GLW_LAB_LD_ROB01:I1.Input[2].5 | BOOL | Robot in T1 Mode |
| LDRobot_T2 | GLW_LAB_LD_ROB01:I1.Input[2].6 | BOOL | Robot in T2 Mode |

Monitor Tags | Edit Tags

*Figure 10: PLC controller tag database in Studio 5000 showing all robot communication tags and their EtherNet/IP mapped addresses*

The complete tag set consists of:

- LDRobot_AtPos1 - Robot at Fluke Position 1 (Digital Input from robot DO)
- LDRobot_AtPos2 - Robot at Fluke Position 2 (Digital Input from robot DO)
- LDRobot_AtFullView - Robot at Full FOV position (Digital Input from robot DO)
- LDRobot_AtHomePos - Robot at home position (Digital Input from robot DO)
- LDRobot_AtQRScanPos - Robot at QR scan position (Digital Input from robot DO)
- LDRobot_CycleComplete - Full scan cycle complete (Digital Input from robot DO)
- LDRobot_StartCycle - Start signal to robot (Digital Output to robot DI)
- LDRobot_Reset - Reset signal to robot (Digital Output to robot DI)
- LDRobot_FaultReset - Fault reset signal to robot (Digital Output to robot DI)
- HMI_Start - Start command from Ignition HMI (Written by Ignition via OPC-UA)
- HMI_Stop - Stop command from Ignition HMI (Written by Ignition via OPC-UA)
- HMI_Reset - Reset command from Ignition HMI (Written by Ignition via OPC-UA)

Robot Digital Outputs (DO) are mapped as PLC Digital Inputs because signals flow from the robot to the PLC, the robot signals its position status and the PLC reads it. PLC Digital Outputs map to robot Digital Inputs for control commands flowing from PLC to robot.

## EtherNet/IP Module Configuration

The FANUC robot was added to the PLC as a Generic Ethernet Module in Studio 5000.

The module configuration required the following parameters:

- IP Address: 10.188.2.186 (robot controller Ethernet port 1)
- Input Assembly Instance: 101 (robot outputs - read by PLC as inputs)
- Output Assembly Instance: 151 (robot inputs - written by PLC as outputs)
- Configuration Assembly Instance: 100
- Input Size: 8 bytes (4 words × 16 bits)
- Output Size: 8 bytes (4 words × 16 bits)
- RPI: 32ms

The module was named GLW_LAB_LD_ROB01 in the PLC project, reflecting the lab location (Galway Lab, Leak Detection, Robot 01). This naming convention ensures traceability when the project is reviewed by other engineers.

## Robot Programming

### Position Register Teaching

The FANUC LR Mate 200iD was programmed using the iPendant teach pendant. Five position registers were taught by physically jogging the robot to each required position and recording the joint angles using the RECORD function on the iPendant:

- PR[1] = HOME POSITION: The robot's default safe resting position, clear of all workpieces
- PR[5] = QR SCAN POSITION: Camera positioned over the QR code on the product
- PR[2] = FLUKE POSITION 1: SV600 positioned at 350mm from the left half of the coil
- PR[3] = FLUKE POSITION 2: SV600 positioned at 350mm from the right half of the coil
- PR[4] = FLUKE FULL FOV: SV600 positioned to capture the full coil width

*Note: PR[5] was assigned to the QR scan position when it was added in Semester 2, after PR[2] through PR[4] had already defined for the acoustic scan positions.*

Initial programming used Linear (L) motion type, which moves the robot end-effector in a straight line between positions. This caused issues because the wrist needed to rotate significantly between positions, and linear motion does not allow wrist rotation during travel. The motion type was changed to Joint (J) motion, which allows each joint to move independently to reach the target position, enabling smooth wrist angle changes during travel and more efficient path execution. Joint motion also results in faster cycle times as each joint takes the most direct angular path to the target location.

### Digital Output Signal Mapping

Digital Output signals were configured on the robot to communicate position status to the PLC. These signals fire at each position to alert the PLC that the robot has reached the target location, which then triggers Ignition to call the Fluke REST API.

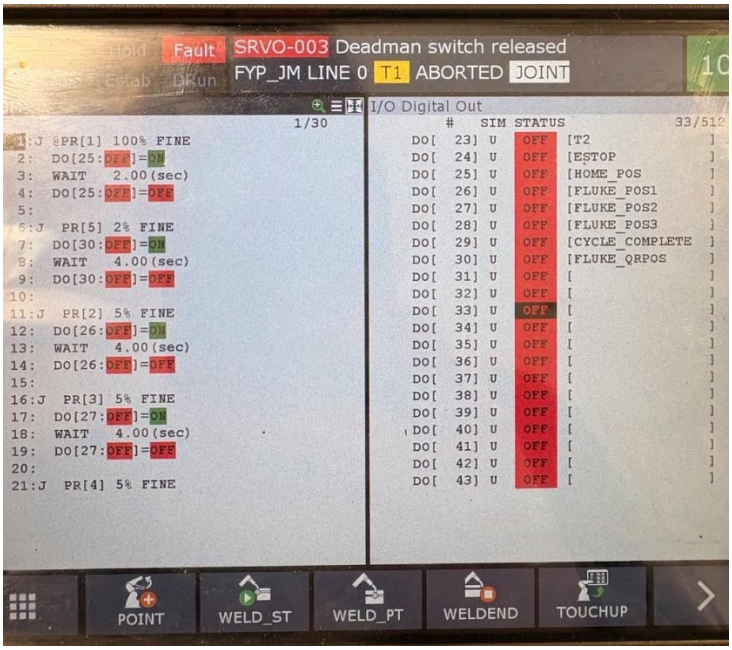


*Figure 11: FANUC robot program on iPendant showing joint motion instructions and digital output signal assignments at each position*

The robot program structure is:

```
1: J @PR[1] 100% FINE          ; Move to home position
2: DO[25:HOME_POS]=ON          ; Signal home position to PLC
3: WAIT  1.00(sec)
4: DO[25:HOME_POS]=OFF
5: J PR[5] 5% FINE             ; Move to QR scan position
6: DO[30:FLUKE_QRPOS]=ON        ; Signal QR position to PLC
7: WAIT  4.00(sec)
8: DO[30:FLUKE_QRPOS]=OFF
9: J PR[2] 5% FINE             ; Move to Position 1
10: DO[26:FLUKE_POS1]=ON       ; Signal Position 1 to PLC
11: WAIT  4.00(sec)
12: DO[26:FLUKE_POS1]=OFF
13: J PR[3] 5% FINE            ; Move to Position 2
14: DO[27:FLUKE_POS2]=ON       ; Signal Position 2 to PLC
15: WAIT  4.00(sec)
16: DO[27:FLUKE_POS2]=OFF
17: J PR[4] 5% FINE            ; Move to Full FOV
18: DO[28:FLUKE_FULLFOV]=ON    ; Signal Full FOV to PLC
19: WAIT  4.00(sec)
20: DO[28:FLUKE_FULLFOV]=OFF
21: J @PR[1] 100% FINE         ; Return to home
```

The 4-second wait at each position allows sufficient time for Ignition to detect the tag change, call the Fluke REST API, receive the JPEG response, extract the event metadata, and complete the MySQL insert before the robot moves to the next position.

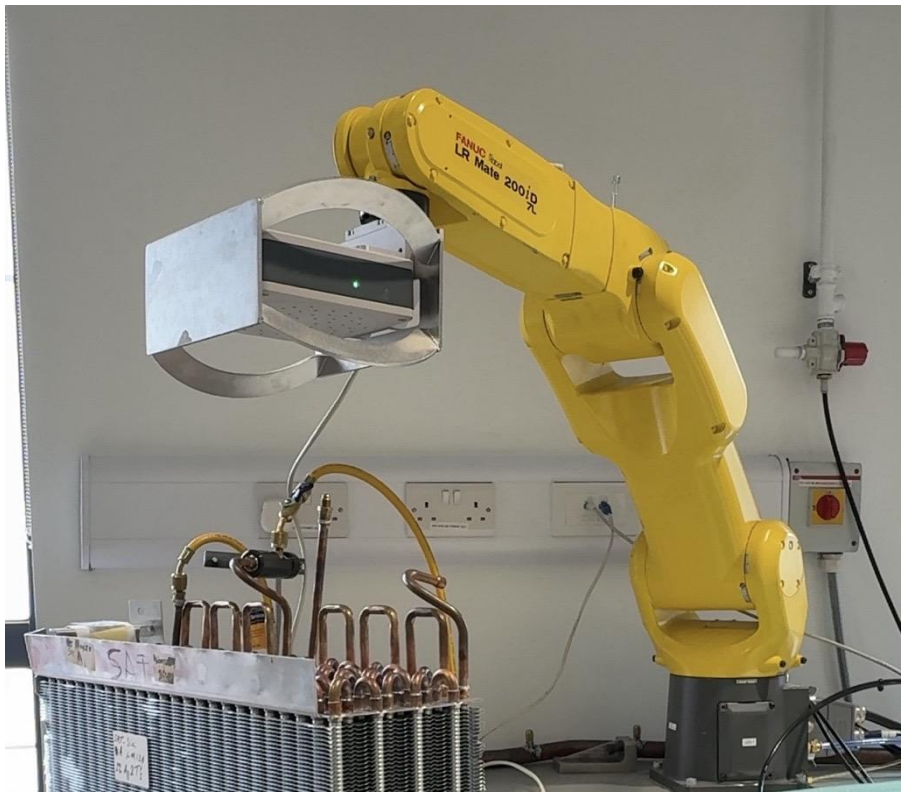


*Figure 12: FANUC LR Mate 200iD robot with Fluke SV600 mounted on end-effector, scanning HVAC coil at Trane Technologies lab (Position 1)*

## EtherNet/IP DI/DO Integration

Establishing verified two-way EtherNet/IP communication between the PLC and robot was the most technically challenging aspect of Semester 1. The validation procedure involved manually toggling output signals and verifying the corresponding input bit state changes on the other side.

Validation was confirmed when Digital Output DO[1] was changed to ON on the robot iPendant, the PLC tag GLW_LAB_LD_ROB01:I.Data[0].0 switched from 0 to 1 in Studio 5000. Conversely, forcing GLW_LAB_LD_ROB01:O.Data[0].0 to 1 in Studio 5000 caused DI[1] to show ON on the robot, confirming bidirectional communication was working correctly.

An operational note: the PLC must be in Remote Run mode (not Program mode) for the EtherNet/IP I/O to be active. The PLC has three modes: Run, Remote, and Program. Remote mode allows switching between Run and Program remotely and is the correct operational mode for this application.

## UOP (User Operator Panel) Configuration

The UOP provides a standardised set of digital I/O signals for remote robot control, allowing the PLC to start, stop, hold, and reset the robot program without physical operator interaction. Configuring the UOP correctly is essential for achieving fully automatic robot operation under PLC control.

The UOP signals use the same EtherNet/IP I/O network as the DI/DO signals but are mapped to a reserved bit range. On the robot, the UOP was configured to:

- Rack: 89 (EtherNet/IP rack)
- Slot: 1
- Start bit: 65 (immediately after the DI/DO range of 1-64)

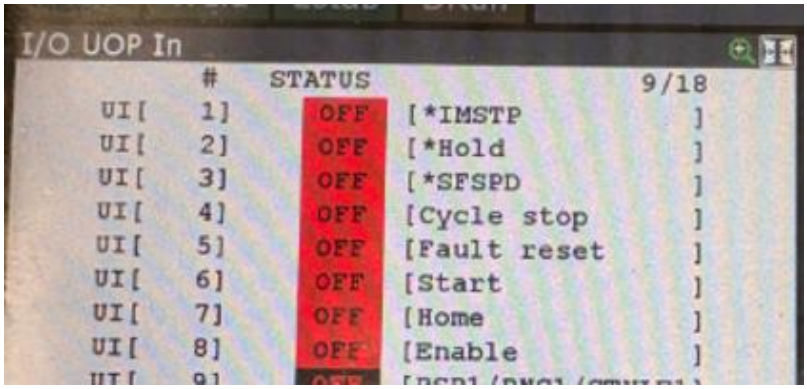


*Figure 13: Robot UOP signal configuration screen on iPendant showing UI/UO signal assignments*

The critical UOP signals and their functions are:

| Signal | Direction | Function |
|---|---|---|
| UI[1] IMSTP | PLC → Robot | Must be ON to indicate no emergency stop condition |
| UI[2] HOLD | PLC → Robot | Must be ON to keep robot active (OFF = pause motion) |
| UI[3] SFSPD | PLC → Robot | Confirms robot is allowed to run at standard speed |
| UI[4] CSTOPI | PLC → Robot | Cycle stop inactive - must be ON to allow execution |

| Signal | Direction | Function |
|---|---|---|
| UI[6] CMD ENBL | PLC → Robot | Allows motion commands from PLC |
| UI[18] PROD START | PLC → Robot | Pulse to start the selected robot program |
| UO[2] FAULT | Robot → PLC | Robot fault active - must be cleared before start |
| UO[6] SYS READY | Robot → PLC | Robot system ready - must be ON before start allowed |
| UO[10] BUSY | Robot → PLC | Robot program is currently running |

*Table 6: Key UOP signal definitions and functions*

## Ladder Logic Development

The PLC ladder logic was structured into a Main Routine and two subroutines: RobotControl and IO_Mapping. The Main Routine calls both subroutines using JSR (Jump to SubRoutine) instructions on every scan cycle. This modular structure makes the logic easy to maintain and debug.

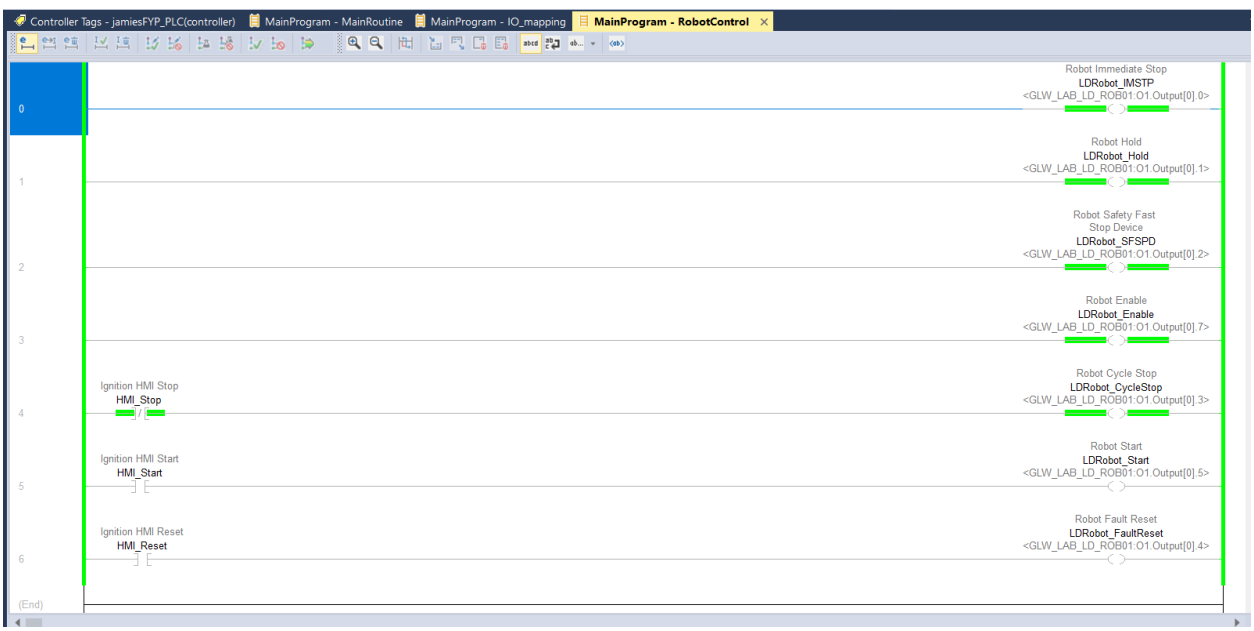


*Figure 14: Studio 5000 ladder logic showing always-on UOP enabling rungs (top) and HMI start/stop control rungs (bottom)*

## Always-On UOP Enabling Rungs

The UOP enabling rungs permanently assert the required UI signals to keep the robot ready for automatic operation. Five coils (LDRobot_IMSTP, LDRobot_Hold, LDRobot_SFSPD, LDRobot_Enable, and LDRobot_CycleStop) are energised unconditionally, meaning they are always ON when the PLC is running. Without correct UOP configuration, the robot will not respond to remote start commands from the PLC and can only be operated manually from the teach pendant, making fully automated operation impossible. If any of these signals drops out for any reason, including a PLC program fault or communication loss, the robot will immediately halt, providing a fail-safe behaviour consistent with industrial safety standards.

## HMI Start Pulse Logic

A key challenge in the ladder logic was generating a momentary start pulse from a sustained HMI button press. The FANUC robot's PROD START input expects a rising edge (OFF→ON→OFF) rather than a sustained signal. If the start bit remains high, the robot controller does not recognise subsequent start events.

This was solved using a TON (Timer On-Delay) and ONS (One-Shot Rising) instruction combination. When HMI_Start goes TRUE (written by Ignition button press), the TON timer begins counting up to a 2-second preset. When the timer's Done bit fires, a ONS creates a single-scan pulse that triggers the PROD START output. At the end, HMI_Start is unlatched back to FALSE. This ensures a clean, momentary start signal regardless of how long the operator holds the button.

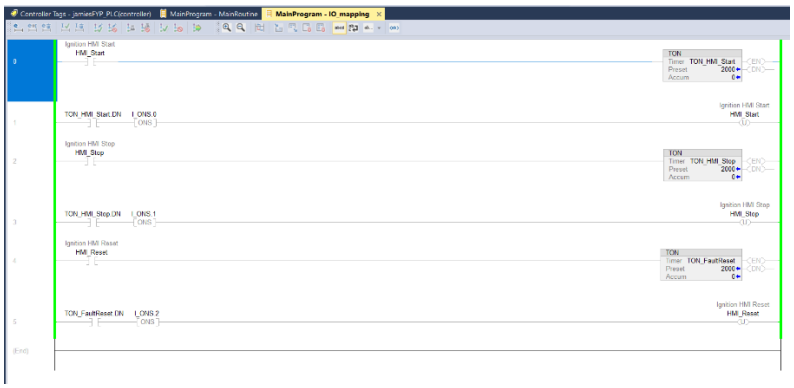


*Figure 15: Studio 5000 ladder logic showing TON and ONS instruction combination for HMI_Start, HMI_Stop, and HMI_Reset*

## MySQL Database Initial Setup

The database was initially prototyped using Microsoft SQL Server in Semester 1, as it was the platform most familiar from previous work experience. The initial table was created on a local SQL Server instance with the following structure:

```
CREATE TABLE capture_images (
    id           INT IDENTITY(1,1) PRIMARY KEY,
    device_name  VARCHAR(50),
    surface_id   INT,
    capture_ts   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    content_type VARCHAR(100),
    img_path     VARCHAR(255)
);
```

However, several constraints became apparent when attempting to deploy this to the Trane Technologies network. IT policy restricted opening new database ports on the corporate network, and the Microsoft SQL Server JDBC driver configuration within Ignition required additional setup that was not supported on the AMT team's existing infrastructure. The decision was therefore made in Semester 2 to switch to MySQL, which was already running on the AMT network and accessible from the Ignition gateway without requiring additional IT approval.

The MySQL schema was redesigned from scratch in Semester 2, using LONGBLOB rather than file paths to store images directly in the database:

```
CREATE TABLE capture_images (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    device_name  VARCHAR(100),
    surface_id   INT,
    capture_ts   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    content_type VARCHAR(100),
    img_blob     LONGBLOB
);
```

An important discovery during the prototype phase was that Ignition can only display images from HTTP URLs rather than local file system paths. This ruled out storing image file paths in the database and instead required storing the full JPEG binary data as a BLOB. A Jython transform script in Ignition reads the BLOB bytes from the database and converts them to a Base64 data URL for display in the Perspective Image component.

This initial schema was progressively extended throughout Semester 2 as additional metadata fields, QR serial number traceability, PASS/FAIL classification, XYZ coordinates, operator notes, and override capabilities were added.

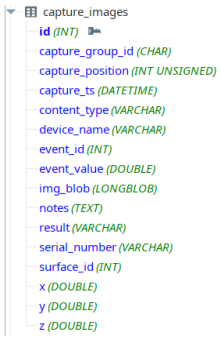


*Figure 16: Final MySQL capture_images table schema*

## Ignition Frontend - Initial Development

The initial Ignition Perspective frontend was developed in parallel with the PLC and database work. The Robot Status page was the first view created, providing start/stop/reset buttons and real-time status indicators for robot operational mode, teach mode, auto mode, and E-Stop state. The status indicators use read-only OPC-UA tag bindings to display the current state of the corresponding PLC tags in real time, updating automatically when the PLC tag value changes without requiring any manual refresh.

Tag bindings in Perspective require bidirectional configuration for control tags. A direct OPC-UA tag binding with bidirectional mode enabled allows Ignition to both read the current PLC tag value and write new values when the operator interacts with the button. A one-directional binding would allow monitoring only. The Start button on the Robot Status page writes TRUE to the HMI_Start tag in the PLC. The PLC ladder logic then generates a momentary pulse from this sustained signal using the TON and ONS instruction combination described in the Ladder Logic section, ensuring the robot receives a clean rising edge trigger rather than a sustained high signal.

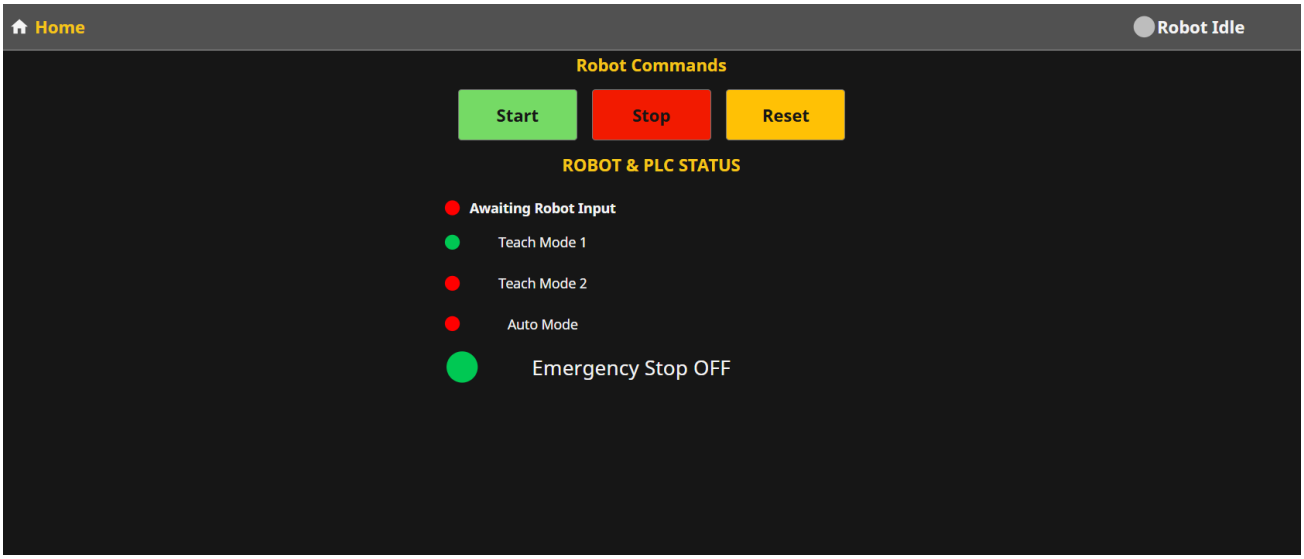


*Figure 17: Ignition SCADA Robot Status and Commands UI*

# Implementation - Semester 2

## Fluke SV600 REST API Research & Validation

Semester 2 began with research and testing of the Fluke SV600 REST API using Postman before any Ignition integration was attempted. This approach allowed the API structure to be fully understood and the correct endpoint parameters to be confirmed before adding the complexity of the Ignition scripting environment.

### API Structure

The SV600 API on port 9011 is organised into three top-level document categories: Sound Source Detection, Authentication, and Device Manager. The Sound Source Detection API is structured into the following endpoint groups:

- actions - Control actions on the device
- areas - Areas of interest configuration
- callbackchannels - Event callback configuration
- data - Primary data retrieval endpoints
- events - Event trigger configuration and results
- measurements - Measurement configuration
- sensors - Sensor channel information
- settings - Device settings
- soundSurfaces - Sound surface management
- status - Device health status
- video - Video streaming endpoints

### Endpoint Validation

Each relevant endpoint was tested in Postman with HTTP Basic Authentication (username: admin, password: admin - factory default credentials). The following endpoints were validated:

- GET /credentials - Returns credential information (200 OK confirmed)
- GET /sensors - Returns 64 MEMS sensor channel details and metadata
- GET /status - Returns device status including deviceStatus: "Ok"
- GET /videoname - Returns video resource identifier for streaming
- GET /data/soundsurface/0/capture - Returns binary JPEG (initially 400 Bad Request due to missing parameters)

The initial failure of the capture endpoint provided an important lesson which was the frequencyVisualizationType parameter is required and must be a valid enum value from the Swagger documentation. The correct value is "Spectrum". After identifying this from the Swagger interface, the full working capture URL was confirmed:

```
GET http://10.188.2.188:9011/data/soundsurface/0/capture
    ?captureFormat=Jpeg
    &frequencyVisualization=Spectrum
    &showDBMarker=true
```

This endpoint returned a 200 OK response with a Content-Type of image/jpeg and a binary JPEG payload. The hex view in Postman confirmed the JPEG magic bytes FF D8 FF at the start of the response, confirming a valid image. These bytes was important because later testing within Ignition would reveal a bug where image bytes appeared valid in size but contained only zeros, making this baseline Postman validation a good reference point for diagnosing that issue.

## Image Capture Pipeline Development

All image capture functions were written as gateway-scoped project library functions in the Ignition Designer, stored under Scripting/Project Library/getImage. Gateway scope was essential as these functions make outbound network requests that are not permitted from client-scoped scripts.

### Authentication Function

All SV600 API calls require HTTP Basic Authentication. A reusable Jython function was created in the Ignition project library to build the required Authorization header:

```
def _basic_auth_header(username, password):
    import base64
    creds = "%s:%s" % (username, password)
    encoded = base64.b64encode(creds.encode("utf-8")).decode("utf-8")
    return "Basic " + encoded
```

### HTTP Download Function with Redirect Handling

A key implementation challenge discovered during initial API testing was that the SV600 sometimes responds with an HTTP redirect before serving the image. Standard HTTP

clients may not re-apply authentication headers when following redirects, causing a 401 Unauthorised response on the redirected request. A custom download function was written to handle this manually:

```
def download_with_basic_auth_follow_redirects(url, auth_header):
    from java.net import URL
    from java.io import ByteArrayOutputStream
    import jarray

    conn = URL(url).openConnection()
    conn.setInstanceFollowRedirects(False)
    conn.setRequestProperty("Authorization", auth_header)
    code = conn.getResponseCode()

    if code in [301, 302, 303, 307]:
        location = conn.getHeaderField("Location")
        conn = URL(location).openConnection()
        conn.setRequestProperty("Authorization", auth_header)
        code = conn.getResponseCode()

    in_stream = conn.getInputStream()
    baos = ByteArrayOutputStream()
    buf = jarray.zeros(4096, "b")  # Java byte[]
    n = in_stream.read(buf, 0, len(buf))
    while n != -1:
        baos.write(buf, 0, n)
        n = in_stream.read(buf, 0, len(buf))

    return code, conn.getContentType(), baos.toByteArray()
```

The function returns three values: the HTTP response code, the Content-Type header, and the raw response bytes as a Java byte array. The response code is checked to confirm a 200 OK before the bytes are processed. The Content-Type confirms the response is image/jpeg. The raw bytes are passed directly to the MySQL BLOB insert.

## Zero-Byte JPEG Bug - Investigation & Resolution

After the image capture function was initially implemented using a Python bytearray buffer, all stored images appeared broken when retrieved from the database. The Base64 encoded image data began with a long sequence of AAAA... characters, which is the Base64 representation of 0x00 bytes.

## Investigation Process

The investigation followed a systematic process of eliminating possible causes:

1. Timing hypothesis: Perhaps the SV600 was not ready when the capture request was sent. A retry loop of 20 attempts with delays was implemented. All 20 attempts returned the same zero-byte result, ruling out timing as the cause.
2. API endpoint hypothesis: The endpoint was tested in Postman independently. Postman returned a valid 205KB JPEG with FF D8 FF bytes confirmed in the hex view. The API was working correctly.
3. Script Console testing: A HTTP GET was written in the Ignition Script Console. The response showed HTTP 200, image/jpeg, ~110kB size, but the first 8 bytes were all 0x00. A real JPEG cannot start with zeros.
4. Java interop hypothesis: The issue was in how Jython handled the Java InputStream. In Jython's Java interop environment, using a Python bytearray as the read buffer for Java's InputStream.read() causes the ByteArrayOutputStream to be filled with zeros even when the stream contains real data.

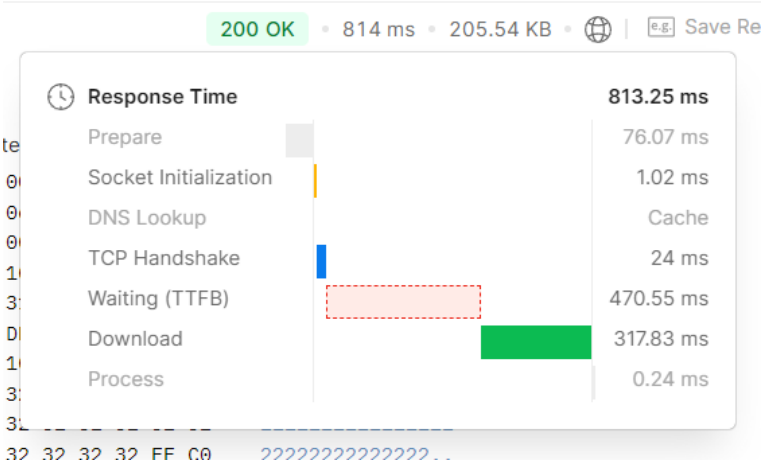


*Figure 18: Postman Response Time to GET image*

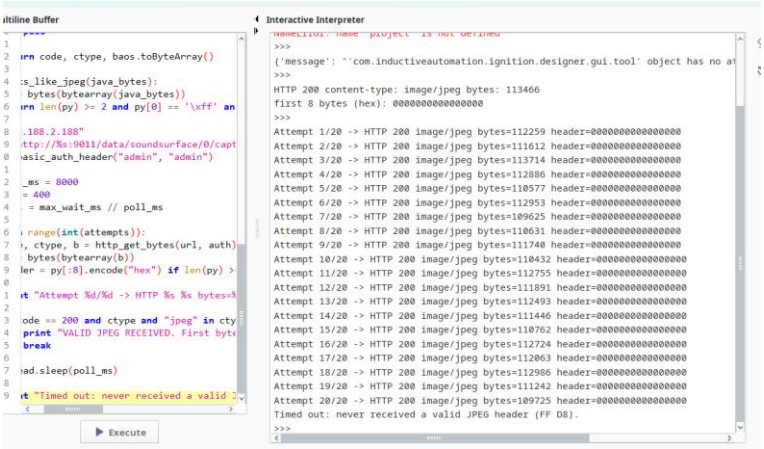


*Figure 19: Script console debugging potential timing issue*

## Resolution

The fix was to replace the Python bytearray buffer with a Java native byte array created using jarray.zeros():

```
# BROKEN - uses Python bytearray
buf = bytearray(4096)
n = in_stream.read(buf, 0, len(buf))

# FIXED - uses Java byte[] jarray
import jarray
buf = jarray.zeros(4096, "b")
n = in_stream.read(buf, 0, len(buf))
```

After this change, the script console confirmed: HTTP 200, image/jpeg, header FF D8 FF, a valid JPEG. This was the breakthrough that unblocked the entire image capture pipeline. This bug was particularly difficult to diagnose because all surface indicators appeared correct - the HTTP request succeeded, the reported content length was accurate, the database insert completed without errors, and the byte count matched. Only direct inspection of the raw bytes revealed the silent data corruption.

## Database Schema Evolution

The capture_images table was extended multiple times throughout Semester 2 as new features were added. The final schema is:

```
CREATE TABLE capture_images (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    device_name       VARCHAR(100),
    surface_id        INT,
    capture_ts        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    content_type      VARCHAR(100),
    img_blob          LONGBLOB,
    capture_position  INT,
    serial_number     VARCHAR(100),
    capture_group_id  VARCHAR(36),
    event_id          VARCHAR(100),
    event_value       FLOAT,
    x                 FLOAT,
    y                 FLOAT,
    z                 FLOAT,
    result            VARCHAR(10),
    notes             TEXT
);
```

Each column addition was performed using ALTER TABLE statements executed from the Ignition Script Console using system.db.runUpdateQuery(). The capture_group_id column was initially added to link all position captures from a single test cycle under one shared UUID, enabling queries that retrieve all images for a specific test in one operation.

During Semester 2, it was identified that each HVAC coil product already had a QR code containing the product serial number. The serial number was implemented as an additional identifier alongside the UUID, decoded automatically by the robot during the scan cycle using the Fluke SV600 camera and ZXing Java library. This removed the need for an operator to manually enter the serial number at the start of each test, reducing manual steps and the risk of human entry errors. The UUID was retained as a guaranteed unique grouping key for cases where the QR decode may not succeed.

## Event Metadata Extraction

Alongside the JPEG image, each capture also retrieves the latest event metadata from the SV600 event results endpoint. An event must be configured on the SV600 with a threshold value. For testing purposes, a threshold of 0 dBSPL was used to ensure the event always triggers and produces results. In a production deployment, this threshold would be set to suit the application.

The metadata extraction function queries the event results endpoint and selects the newest record by timestamp:

```
def fetch_latest_event_metadata(ip, event_id, auth):
    url = "http://%s:9011/data/event/%s?duration=1000000000&amount=10"
           % (ip, event_id)
    code, ctype, raw_bytes =
download_with_basic_auth_follow_redirects(url, auth)
    if code == 200:
        import json
        data = json.loads(str(raw_bytes.tostring(), "utf-8"))
        if not data: return None
        newest = max(data, key=lambda r: r.get("timestamp", ""))
        return {
            "event_id":    newest.get("id"),
            "event_value": newest["values"][0] if newest.get("values")
else None,
            "x": newest["locations"][0]["x"] if newest.get("locations")
else None,
            "y": newest["locations"][0]["y"] if newest.get("locations")
else None,
            "z": newest["locations"][0]["z"] if newest.get("locations")
else None,
        }
    return None
```

The returned dictionary provides the event_id for database reference, event_value as the dB SPL measurement used for PASS/FAIL classification, and the x, y, z coordinates representing the estimated acoustic source location in the sensor's local coordinate frame. If the endpoint returns an empty array, indicating the event threshold was not met. The function returns None and the image row is stored without metadata, classified as UNKNOWN rather than PASS or FAIL.QR Code Traceability

## ZXing Library Selection & Deployment

QR code decoding in Ignition requires a Java barcode library because Jython runs on the JVM and can directly use Java classes. ZXing (Zebra Crossing) was selected because it is the most widely used open-source Java barcode library, distributed as lightweight JARs (core-3.5.3.jar and javase-3.5.3.jar, approx. 700KB), and supports decoding directly from Java BufferedImage objects.

The correct deployment location for Java libraries in Ignition is the gateway classpath directory:

```
C:\Program Files\Inductive Automation\Ignition\lib\core\gateway
```

The Ignition Gateway service must be restarted after adding JARs to this location. A Gateway Timer Script was used to verify successful loading by attempting the imports and logging the result:

```
try:
    from com.google.zxing.client.j2se import
BufferedImageLuminanceSource
    system.util.getLogger("ZXingTest").info("ZXing javase import OK on
Gateway")
except:
    system.util.getLogger("ZXingTest").info("ZXing import FAILED on
Gateway")
```

| Logger | Time | Message |
|---|---|---|
| I ZXingTest | 24Mar2026 17:48:37 | Zxing javase import OK on Gateway |
| I ZXingTest | 24Mar2026 17:48:32 | Zxing javase import OK on Gateway |
| I ZXingTest | 24Mar2026 17:48:27 | Zxing javase import OK on Gateway |

*Figure 20: Gateway Logger receiving updates from Timer Script execution*

Deployment to the lab gateway required IT department involvement due to admin rights restrictions. The full escalation process is documented in the Challenges section.

## QR Decode Implementation

Once ZXing was confirmed loaded, the decode function was implemented as a gateway-scoped project library function:

```
def decode_qr_from_image_bytes(img_blob):
    from java.io import ByteArrayInputStream
    from javax.imageio import ImageIO
    from com.google.zxing import MultiFormatReader, BinaryBitmap
    from com.google.zxing.client.j2se import
BufferedImageLuminanceSource
    from com.google.zxing.common import HybridBinarizer

    bais = ByteArrayInputStream(img_blob)
    buffered_image = ImageIO.read(bais)
    if buffered_image is None: return None

    source = BufferedImageLuminanceSource(buffered_image)
    bitmap = BinaryBitmap(HybridBinarizer(source))
    reader = MultiFormatReader()

    try:
        result = reader.decode(bitmap)
        return result.getText()
    except:
        return None
```

## Capture Group ID & Serial Number Flow

The serial number decoded at Position 0 (QR scan) must persist across all subsequent position captures (1, 2, Full FOV) to link all images for a single product test. This was implemented using Ignition memory tags to store the serial number and capture group UUID between tag event script executions:

- At Position 0: Generate a new UUID (java.util.UUID.randomUUID()), store it in the memory tag [default]LD_Robot_HMI/CurrentGroupId. Capture QR image, decode serial number, store in [default]LD_Robot_HMI/CurrentSerialNumber.
- At Positions 1, 2, Full FOV: Read the UUID and serial number from memory tags. Include them in the MySQL INSERT.
- After Full FOV capture: Write empty string to both memory tags to clear them for the next test cycle.

If the QR decode returns None – for example if the QR code is obscured, damaged, or out of frame – the serial number memory tag is left empty and the capture group UUID still groups the position captures together. This ensures that the test cycle continues and results are still stored, with the serial number field left NULL in the database rather than causing an entire test to fail.

## PASS/FAIL Classification

Automated PASS/FAIL classification is calculated at capture time using the dBSPL event value returned by the Fluke API. The classification logic in the capture function is:

```
fail_db_threshold = 10.0  # Configurable

if event_value is None:
    result = "UNKNOWN"
elif float(event_value) >= fail_db_threshold:
    result = "FAIL"
else:
    result = "PASS"
```

The threshold value is configurable and would be set to the appropriate production threshold by Trane Technologies engineers before deployment. During development, a low threshold of 10.0 dBSPL was used for testing purposes.

## Ignition UI - Leak Visualisation Review

The Leak Visualisation Review page is the primary result-viewing interface for operators. It consists of:

- A timeframe dropdown (30 mins, 1 hour, 2 hours, 1 day, 2 days, None) that filters the results list
- A Capture Set dropdown populated by a Named Query returning serial numbers with their timestamps.
- Three embedded PositionReview views, one per scan position, each displaying the JPEG image, dB SPL value, X/Y/Z coordinates, and PASS/FAIL badge
- A notes text area for operator comments linked to the selected serial number and position
- PASS and FAIL override buttons that write back to the database

Each embedded PositionReview view receives the serial_number and position number as parameters. Embedded views were used for each position panel rather than duplicating the layout three times, as they allow the same PositionReview view template to be reused with different position parameters. This makes the UI scalable if additional scan positions are added in future, a new embedded view instance can be added without redesigning the page. A Named Query retrieves the image and metadata for that specific group/position combination. A Jython transform script on the image component converts the BLOB bytes to a Base64 data URL:

```
def transform(self, value, quality, timestamp):
    py = system.dataset.toPyDataSet(value)
    if len(py) == 0: return ""
    img_data = py[0]["img_blob"]
    if img_data is None: return ""
    import base64
    b64 = base64.b64encode(bytes(img_data))
    return "" + b64.decode("utf-8")
```

## Operator Override & Notes Feature

The operator override feature allows operators to manually correct automated PASS/FAIL results, for example after a joint has been reworked and re-inspected. Two buttons (PASS and FAIL) each trigger a runAction script that:

1. Reads the currently selected serial number from the Capture Set dropdown
2. Reads the currently selected position from a session custom property
3. Executes a Named Query (changeResult) to UPDATE the result column in the database

Notes are implemented similarly with a Named Query (addNotes) that updates the notes TEXT column for the selected serial number and position. Notes are retrieved on dropdown change by a separate Named Query returning the most recent note for that combination, ordered by capture_ts descending.

## Fluke SV600 Coordinate System Research

During Semester 2, a technical meeting was arranged with Martin Budweg, Fluke R&D Manager, to clarify the nature of the X/Y/Z coordinates returned by the SV600 REST API. This meeting was prompted by uncertainty about whether the coordinates could be used directly to guide the robot to zoom in on a detected leak.

Key findings from the Fluke R&D meeting:

- The SV600 XYZ output represents a sensor-local Cartesian coordinate system derived from beamforming across the 64 MEMS microphone array. The coordinates indicate the direction and estimated depth of the acoustic source relative to the sensor's own reference frame.
- The coordinates are not pixel coordinates in the camera image, or world/factory coordinates. They are beamforming-derived direction vectors in the sensor's local frame.
- The coordinates change whenever the robot changes position, because moving the robot changes the sensor's reference frame relative to the product. A frame transform is required to map sensor-local coordinates to robot/world coordinates.
- The distance setting configured on the SV600 device directly affects the coordinate output. If the configured distance does not match the physical distance, the X/Y/Z values will be offset.

This finding confirmed that direct robot guidance using raw SV600 coordinates is not straightforward and requires implementing a sensor-to-robot coordinate frame transform as a future development item as implementing the transform required to map these coordinates correctly to the robot frame would be outside the remaining project timeline. The coordinates are still stored in the database as they provide valuable data for future analysis and potential ML applications.

# Testing & Validation

## EtherNet/IP Communication Validation

EtherNet/IP communication between the PLC and robot was validated by manual bit forcing in Studio 5000 and comparison with the robot I/O monitor on the iPendant:

- Forcing GLW_LAB_LD_ROB01:O.Data[0].0 to TRUE in Studio 5000 → DI[1] shows ON on robot
- Setting DO[1] to ON on robot iPendant → GLW_LAB_LD_ROB01:I.Data[0].0 shows TRUE in Studio 5000
- UOP signals validated by observing UI bit states on the robot changing in response to PLC output forcing

## Image Capture Pipeline Validation

The image capture pipeline was validated using the Ignition Script Console with a known test product. The validation procedure:

1. Call capture_image_and_store() from the Script Console with the SV600 IP address
2. Check Script Console output for HTTP response code (expected 200) and byte size (expected 100-200KB)
3. Query capture_images table in MySQL to confirm a new row was inserted
4. Refresh the Leak Visualisation Review page and verify the image displays correctly
5. Confirm the JPEG bytes are FF D8 FF by checking the first bytes of the transformed data from the img_blob column in the database

## QR Decode Pipeline Validation

The QR decode pipeline was validated using a test image containing a known QR code (encoded text: "TEST_QR_1"). The test image was inserted into the database, and the decode function was called from a Gateway Timer Script:

```
result = project.getImage.decode_qr_from_image_bytes(img_bytes)
system.util.getLogger("ZXingDecode").info("QR decoded text: " +
str(result))
```

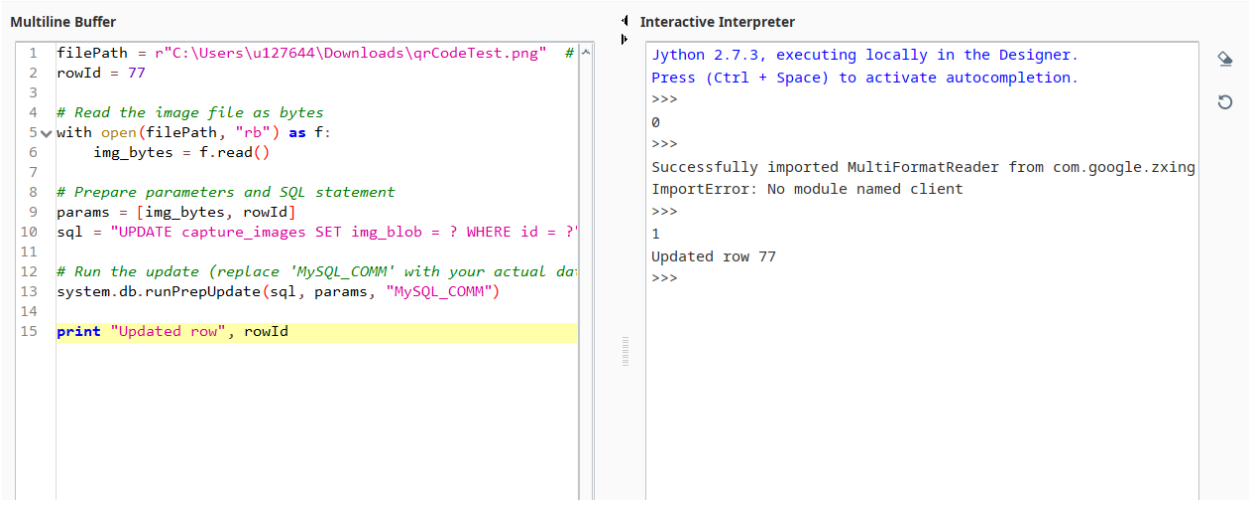


*Figure 21: Script Console for adding generated QR Code to the database for testing*

Gateway logs confirmed: "QR decoded text: TEST_QR_1". This validated the full decode pipeline from byte array to decoded string.

A second validation test was performed to confirm the QR decode pipeline functioned correctly within the automated tag event flow, prior to testing with the live SV600 camera. A test mode flag was implemented using a dedicated Ignition memory tag (LDRobot_TestMode). When this tag is set to TRUE, the tag event script triggered by the robot position change substitutes the live SV600 REST API capture with a pre-stored test image retrieved directly from the database. The ZXing decode function then processes this image, extracts the serial number, and writes it to the CurrentSerialNumber and CurrentGroupId memory tags - replicating the exact behaviour of a live QR scan without requiring the robot to be present. This approach allowed the full end-to-end tag event flow to be validated remotely before lab testing.

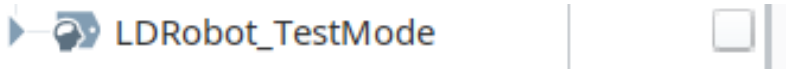


*Figure 22: Tag for testing QR decode script*

*Figure 23: Memory tags for saving UUID and decoded Serial Number*

## End-to-End System Test

A complete end-to-end system test was performed using a live HVAC coil product at the Trane Technologies AMT lab. The test sequence:

1. Operator pressed Reset then Start on the Ignition HMI Robot Status and Commands page
2. Ignition wrote HMI_Start=TRUE to PLC using the ControlLogix EtherNet/IP driver
3. PLC ladder logic confirmed robot at home, pulsed PROD START to robot
4. Robot moved to QR scan position, DO[30] fired, Ignition captured QR image, ZXing decoded serial number successfully
5. Operator pressurised the HVAC coil with nitrogen before acoustic scanning began
6. Robot moved to Position 1, DO[27] fired, Ignition captured acoustic heatmap, dBSPL and coordinates retrieved, stored to MySQL
7. Robot moved to Position 2, DO[28] fired, repeated capture and storage
8. Robot moved to Full FOV, DO[29] fired, final capture and storage, memory tags cleared
9. Robot returned to home
10. Ignition Leak Visualisation Review displayed three position images with PASS/FAIL classifications

The test completed successfully. All three position images were stored and retrieved correctly. The serial number decoded from the QR code was visible in the database linked to all three capture rows. PASS/FAIL classification was applied automatically based on the dB SPL threshold.

# Ethics & Sustainability

This project was developed with consideration for ethical responsibilities and sustainable engineering practice in an industrial context.

Safety was the primary ethical concern given the industrial setting. The system was designed with multiple safety interlocks, the PLC verifies the robot is at the home position before enabling the start sequence, an E-Stop input is monitored continuously, and the robot only operates in automatic mode when the PLC has explicitly enabled the UOP signals. No new safety risks were introduced by the automation, as the robot cell already existed with its own safety barriers.

Data privacy was considered in the design of the traceability system. Product serial numbers stored in the MySQL database are industrial product identifiers and contain no personal data. Access to the database is restricted to the Ignition gateway network and is not exposed externally.

From a sustainability perspective, the project directly contributes to reduced energy waste at Trane Technologies. Early leak detection at the manufacturing stage prevents defective products from reaching customers, where refrigerant leakage would contribute to greenhouse gas emissions. The system also reduces the labour time required for inspection, freeing operators for higher-value tasks.

The replacement of helium with nitrogen as the test gas also represents a sustainability improvement. Helium is a non-renewable resource extracted as a by-product of natural gas production and cannot be synthesised. Nitrogen makes up 78% of the atmosphere and can be produced locally through air separation, making it significantly more sustainable and cost-effective as a test gas.

The digital traceability system enables future trend analysis and predictive quality control. By accumulating historical data on which products fail, which production shifts have higher failure rates, and which specific coil regions are most prone to leaks, the manufacturing process can be continuously improved to reduce defects at source.

# Results

## System Performance Summary

| Parameter | Before (Manual) | After (Automated) |
|---|---|---|
| Inspection method | Manual, operator-dependent | Fully automated, zero manual steps |
| Detection distance | ~1400mm (variable) | 350mm (consistent, optimised) |
| Frequency range | Broadband, unfiltered | 48-51kHz targeted, noise-free |
| Scan positions per test | 1 (single pass) | 4 (QR + 3 acoustic positions) |
| Product traceability | None | QR serial number → MySQL |
| Result classification | Operator visual judgement | Automated PASS/FAIL per position |
| Historical data | None | Full audit trail - image BLOBs, dBSPL, XYZ, UUID, serial number |
| Operator notes | Unavailable | Per-position text notes in database |
| Result override | Unavailable | Manual PASS/FAIL override via HMI |

*Table 7: Before and after comparison of all key system parameters*

## Technical Achievements

The following specific technical achievements were delivered:

- End-to-end EtherNet/IP communication validated between Allen-Bradley 1756-L71 PLC and FANUC LR Mate 200iD robot, with confirmed bidirectional DI/DO and UOP signal exchange.
- Event-driven REST API image capture from the Fluke SV600, triggered by PLC tag change events detected in Ignition, with zero manual operator steps beyond initiating the test.
- JPEG binary blob storage in MySQL using Jython scripting in the Ignition gateway, with successful retrieval and display in the Perspective UI using Base64 transform scripts.
- ZXing Java library successfully deployed to the Ignition gateway and validated for QR code decoding from image bytes, with decoded serial numbers persisted to MySQL.
- Automated PASS/FAIL classification per scan position based on configurable dB SPL threshold, with operator override capability allowing manual reclassification of results following rework or re-inspection.
- dB SPL, X/Y/Z acoustic source coordinates, and JPEG images stored per capture, all linked to the decoded QR serial number and a shared capture UUID.

- System successfully validated on live HVAC coil products at the Trane Technologies AMT lab, Galway, with complete end-to-end cycle execution confirmed including QR decode, three-position acoustic capture, MySQL storage, and PASS/FAIL classification.

# Challenges & Solutions

## Challenge 1: EtherNet/IP I/O Word Mismatch (Semester 1)

### Problem

When establishing the EtherNet/IP connection between the PLC and the FANUC robot controller, the robot immediately displayed two fault codes: "EtherNet/IP Adapter Error" and "Port Assignment Invalid". The module showed as OFFLINE in both the Studio 5000 module tree and the robot's EtherNet/IP adapter screen. The robot DI/DO signals were not reflecting any changes made to the PLC outputs.

### Investigation

The root cause was traced to a mismatch between the I/O sizes configured on the robot and the PLC. The robot controller was configured for 4 words of input and 4 words of output. A word on the FANUC controller is 16 bits. Therefore, 4 words = 4 × 16 = 64 bits per direction. On the PLC side, the Generic Ethernet Module was initially configured with incorrect byte sizes that did not match this 64-bit configuration. Additionally, the DI/DO signals were mapped to incorrect bit ranges that overlapped with the UOP signal range.

### Resolution

The PLC Generic Ethernet Module was reconfigured with 8 input bytes and 8 output bytes (8 × 8 bits = 64 bits per direction, matching the robot's 4 × 16-bit word configuration). The DI/DO tag bit assignments were corrected to use the range 1-64 bits for standard process signals, leaving bits 65-80 exclusively for UOP signals. After downloading the updated PLC program and restarting the robot to apply the I/O configuration changes, the module showed as RUNNING and bidirectional communication was confirmed.

## Challenge 2: Zero-Byte JPEG BLOB in Jython (Semester 2, Week 4)

### Problem

All images captured from the Fluke SV600 and stored in MySQL appeared as broken images when displayed in Ignition. Retrieving the blob from the database and decoding it to Base64 produced a long string of AAAA... characters, indicating all zero bytes. The DB inserts were completing without errors, and row counts were correct, but the image data itself contained no valid pixel information.

### Investigation

The Fluke API was confirmed working correctly using Postman, returning valid JPEG data with FF D8 FF bytes. The same endpoint called from the Ignition Script Console returned HTTP 200 and the correct byte count, but the first 8 bytes were all 0x00. A retry loop of 20 attempts with delays ruled out timing as the cause, pointing to a Jython/Java interop issue with the binary read buffer.

### Resolution

The root cause was identified as Jython's handling of Java's InputStream.read() method. When a Python bytearray is passed as the buffer parameter, Jython's Java interop layer fills the ByteArrayOutputStream with zeros rather than the actual stream data. Replacing the Python bytearray with a Java-native byte array using jarray.zeros(4096, "b") resolved the issue. The Java byte[] integrates correctly with Java's InputStream.read(), and the ByteArrayOutputStream received the actual image bytes. The fix was confirmed by checking the first two bytes of the response for the FF D8 JPEG byte sequence.

## Challenge 3: ZXing Java Library Deployment (Semester 2, Weeks 7-9)

### Problem

Implementing QR code decoding in Ignition required deploying the ZXing (Zebra Crossing) Java library (two JAR files: core-3.5.3.jar and javase-3.5.3.jar) to the Ignition gateway's classpath. The correct location is C:\Program Files\Inductive Automation\Ignition\lib\core\gateway, and the gateway service must be restarted after file placement. However, the lab gateway PC was protected by IT department admin restrictions that prevented direct file system access.

### Investigation & Escalation

Multiple attempts were made to resolve the access issue:

1. First contact with a senior engineer who could not assist - local admin rights required
2. Contact with Trane Technologies IT department - initially stated they would resolve by Thursday, then no response
3. Escalation to TK manager - recommended contacting an intern with local lab PC access
4. Contact with Senior Software Engineer - encountered admin rights block on copy operation
5. Parallel escalation to Ignition standards engineer - confirmed JARs were safe for deployment to Ignition.
6. IT department eventually granted access to the intern after approval by the IT manager

### Resolution

After two weeks of escalation, IT department admin credentials were used to copy the ZXing JARs to the correct Ignition classpath directory. The Ignition Gateway Windows service was restarted using the services.msc. A Gateway Timer Script confirmed successful library loading by logging "ZXing javase import OK on Gateway" repeatedly in the gateway logs.

## Challenge 4: Fluke SV600 Coordinate System Clarification (Semester 2, Week 8)

### Problem

The X/Y/Z coordinate values returned by the SV600 event results endpoint were not clearly documented in the available documentation. It was unclear whether they represented pixel coordinates in the image, real-world factory coordinates, or some other reference frame. This made it impossible to determine whether the coordinates could be used directly to guide the robot for a close-up zoom-in inspection feature.

## Resolution

A technical meeting was arranged directly with Martin Budweg of the Fluke R&D engineering team. The meeting confirmed that the coordinates are sensor-local Cartesian outputs from the beamforming algorithm, representing the direction and estimated depth of the acoustic source in the sensor's own reference frame. They are not pixel coordinates and not world coordinates. A frame transform using the robot's current pose and the sensor's mounting offset would be required to map these to robot coordinates. This finding redirected the zoom-in feature to future work. The complexity of implementing and validating a sensor-to-robot coordinate frame correctly was determined to exceed what was achievable within the remaining two-week sprint. With the coordinate data being stored in MySQL to support future implementation without changes to the data pipeline.

# Conclusion

## Project Summary

This project successfully delivered a fully automated acoustic leak detection system for brazed copper HVAC joints, replacing a previously manual, untracked, and inconsistent inspection process at Trane Technologies, Galway. The system integrates five industrial components - a FANUC LR Mate 200iD robot, Allen-Bradley ControlLogix PLC, Fluke SV600 acoustic sensor, Ignition SCADA, and MySQL database into a single, operator-initiated automated pipeline.

All primary project objectives were met. Automated multi-position scanning, event-driven image capture via REST API, QR-based serial number traceability, automated PASS/FAIL classification, full data persistence to MySQL, and a functional operator-facing SCADA interface. The system was validated on live HVAC coil products in the Trane Technologies AMT lab.

The project demonstrated the full range of skills developed across the B.Eng. in Software and Electronic Engineering: Internet Technologies (EtherNet/IP), software engineering (Jython scripting, REST API integration, database design, frontend development), system integration, fault investigation, and professional working practices in an industry environment.

## Reflection

The most valuable learning from this project was the practical complexity of integrating multiple industrial systems. Each individual component is well-documented in isolation, but the interactions between required a level of low-level investigation that goes beyond anything done in this course. The ability to systematically narrow down root causes using logging, hex inspection, and controlled experiments was the most important skill applied throughout the project.

The ZXing deployment challenge highlighted the importance of stakeholder management and escalation in an industry environment. The technical solution was straightforward once access was granted, but the two-week delay required navigating multiple layers of approval across IT, engineering, and management. This is a realistic challenge in any

industrial software project and was valuable experience for professional engineering practice.

## Future Work

The following features were identified during development for future implementation:

- Fluke SV600 to Robot Coordinate Frame Transform: Implement the mathematical transform from sensor-local XYZ coordinates to robot world frame coordinates, enabling the robot to autonomously reposition for close-up inspection of detected leak locations (zoom-in feature), this would be valuable for large products to get an accurate location on the product.
- Nitrogen Pressure Monitoring: Integrate a solenoid valve with the PLC to automate the product pressurisation step, with a pressure sensor providing a signal to the PLC when the product reaches the required pressure before the scan cycle begins.
- Machine Learning Classification: Use the accumulated historical dB SPL and coordinate data in MySQL to train a classification model for leak severity (small, large, critical) beyond the simple threshold-based PASS/FAIL system.
- AI-Assisted Trend Analysis: Implement an AI chatbot interface (as discussed with the AMT team) that allows engineers to query historical leak data in natural language to identify trends, correlate leaks with production shifts, and target process improvements.
- AI U-Bend Joint Identification and Closed-Loop Brazing Integration: Train a computer vision or acoustic classification model on captured acoustic images and XYZ coordinate data to identify which specific copper U-Bend joint a detected leak originates from. This feeds directly into a separate ongoing project at the Trane Technologies lab, using a robotic induction brazing system – the failed joint ID, serial number and coordinates are written to MySQL database, the brazing robot reads the joint ID and automatically re-brazes the exact failed joint, and the product returns to this system for a re-test to confirm the repair. This creates a fully closed-loop quality control system with no manual rework intervention. The training data pipeline is already in place – acoustic images, dB SPL values and XYZ coordinates are accumulated with every product test.

# References

[1] Inductive Automation. (2024). Ignition SCADA Platform Documentation. Retrieved from: https://docs.inductiveautomation.com

[2] Inductive Automation. (2024). Project Library - Gateway Scope Notes. Ignition User Manual. Retrieved from:

https://www.docs.inductiveautomation.com/docs/8.1/platform/scripting/scripting-in-ignition/project-library

[3] Rockwell Automation. (2023). ControlLogix 5570 Controller User Manual, Publication 1756-UM001. Milwaukee: Rockwell Automation. Retrieved from:

https://literature.rockwellautomation.com/idc/groups/literature/documents/um/1756-um001_-en-p.pdf

[4] Rockwell Automation. (2023). EtherNet/IP Module in Logix 5000 Systems User Manual, Publication 1756-UM003. Milwaukee: Rockwell Automation. Retrieved from:

https://literature.rockwellautomation.com/idc/groups/literature/documents/um/1756-um003_-en-p.pdf

[5] FANUC America Corporation. (2024). LR Mate 200iD Series Mechanical Unit Operator's Manual. Rochester Hills: FANUC America. Retrieved from:

https://www.ecorfan.org/libros/Programming_of_Industrial_robot_Fanuc_LR/Programming_of_Industrial_robot_Fanuc_LR.pdf

[6] FANUC America Corporation. (2024). R-30iA/R-30iB EtherNet/IP Adapter Operator's Manual. Rochester Hills: FANUC America. Retrieved from:

https://www.scribd.com/document/596987551/R30ib-Ethernet-Ip-Operator-Manual

[7] FANUC America Corporation. (2024). FANUC Robot Series - DI/DO I/O Configuration Guide. Rochester Hills: FANUC America.

[8] Fluke Process Instruments. (2024). SV600 Series Fixed Acoustic Imager User Manual, Rev 2.10. Berlin: Fluke Process Instruments. Retrieved from:

https://www.flukeprocessinstruments.com/sites/default/files/sv600_ma_5306246_eng_rev2.10.pdf

[9] ZXing Authors. (2023). ZXing (Zebra Crossing) Java Barcode Library v3.5.3. Retrieved from: https://github.com/zxing/zxing

Maven Central: https://repo1.maven.org/maven2/com/google/zxing/core/3.5.3/

[10] control.com. (2024). FANUC Robot EtherNet/IP Tutorial: Setting Up PLC I/O Communication. Retrieved from: https://control.com/technical-articles/fanuc-robot-ethernet-tutorial-setting-up-plc-i-o-communication/

[11] Inductive Automation Forums. (2024). Importing and using 3rd party Java JAR libraries in Ignition. Retrieved from: https://forum.inductiveautomation.com/t/importing-and-using-3rd-party-java-jar-libraries-in-ignition/11342

[12] Inductive Automation Forums. (2022). View images from database. Retrieved from: https://forum.inductiveautomation.com/t/view-images-from-database/22626

[13] GeeksForGeeks. (2024). How to Generate and Read QR Code with Java using ZXing Library. Retrieved from: https://www.geeksforgeeks.org/java/how-to-generate-and-read-qr-code-with-java-using-zxing-library/

[14] Solis PLC. (2024). Mapping and Configuring FANUC Robot I/O Points. Retrieved from: https://www.solisplc.com/tutorials/mapping-and-configuring-fanuc-robot-io-points

[15] oneRobotics. (2016). Starting FANUC Robots in Auto Mode. Retrieved from: https://www.onerobotics.com/posts/2016/starting-fanuc-robots-in-auto/

[16] UE Systems. (2024). Ultraprobe 9000 Ultrasonic Inspection System. Elmsford: UE Systems Inc. Retrieved from: https://www.uesystems.com/leak-detection/ultraprobe-9000/